

ELEC 392: Principles of Design and Development

Summative Report for the Development of an Autonomous Taxi Prototype to Enhance Public Road Safety in Quackston



Prepared For: Eric Godden

Duclair 2

SD Mystery Incorporated

Rafael Costa (XX), Ishaan Grewal (XX), Nolan Su-Hackett (XX), Declan Smith (XX)

April 6, 2026

Statement of Originality

Our signatures below attest that this submission is our original work.

Following professional engineering practice, we bear the burden of proof for original work. We have read the Policy on Academic Integrity posted on the Faculty of Engineering and Applied Science website (<http://engineering.queensu.ca/policy/Honesty.html>) and confirm that this work is in accordance with the Policy.

Individual 1:

Signature:	<i>Rafael Costa</i>	Date:	April 6, 2026
Name:	Rafael Costa	ID #:	XX

Individual 2:

Signature:	<i>Declan Smith</i>	Date:	April 6, 2026
Name:	Declan Smith	ID #:	XX

Individual 3:

Signature:	<i>Nolan Su-Hackett</i>	Date:	April 6, 2026
Name:	Nolan Su-Hackett	ID #:	XX

Individual 4:

Signature:	<i>Ishaan Grewal</i>	Date:	April 6, 2026
Name:	Ishaan Grewal	ID #:	XX

Executive Summary

This report details the design, development, and evaluation of an autonomous taxi prototype aimed at improving public road safety in Quackston. The project focuses on integrating PID and lane following, computer vision, path finding and graphing algorithms and machine learning models to enable reliable autonomous navigation, lane keeping, and dynamic obstacle detection.

Developed as part of the ELEC 392 Principles of Design and Development course, the project addresses the complex engineering challenge of building a reliable, fully integrated autonomous vehicle under strict computational, physical, and financial constraints. The primary objective was to create a vehicle capable of navigating roadways, responding to traffic signs, avoiding obstacles, and executing passenger pickups and drop-offs while strictly prioritizing safety.

Built upon the PiCar-X and powered by a Raspberry Pi 5, the system integrates three core independent subsystems: Perception, Planning and Localization, and Control. Communication between these processes is maintained through a TCP server. The technical architecture allocates specific responsibilities to each subsystem to ensure optimal performance. The Perception subsystem utilizes a Raspberry Pi Camera and a Google Coral USB Accelerator to process visual data, enabling real-time lane tracking, cross-track error trajectory generation, and object detection for identifying duck passengers and traffic signs. The Planning and Localization subsystem employs Dijkstra's algorithm and interfaces with the competition's Vehicle Positioning and Fare System (VPFS) to compute optimal routes across the city map. Finally, the Control subsystem translates this data into physical actuation, managing PID-based lane keeping, intersection handling, obstacle avoidance, and stopping mechanisms.

Ethically, the design process was guided by a safety-first methodology, meaning safety obligations were consistently prioritized over speed or competition scores. Passenger safety led to the development of a secure Cabin and Restraint Equipment (CARE) system, featuring an elastic band seat belt to protect the duck passengers during sudden stops or turns. To ensure predictable behavior on shared roads, the vehicle was purposefully engineered to operate at lower speeds, which improved the reliability of turns and allowed sufficient processing time for the hardware. Furthermore, the autonomous taxi complies with all competition mandates, including emergency stopping capabilities, audio-visual signaling for pedestrians, and a non-invasive hardware mounting approach utilizing the Duck Deck.

Overall, the design was a successful instance of an autonomous vehicle, scoring success at competition and navigating the map of Quackston with only some errors (although issues with the Raspberry Pi prevented the team from completing multiple fares). The team was able to successfully integrate several complex subsystems into one working final product that successfully completed most project goals as set out by the ELEC 392 teaching team.

Table of Contents

ELEC 392: Principles of Design and Development	i
Summative Report for the Development of an Autonomous Taxi Prototype to Enhance Public Road Safety in Quackston	i
Statement of Originality	ii
Executive Summary	iii
List of Figures	vi
List of Tables	vi
1. Introduction	1
1.1 Background	1
1.2 Problem Statement.....	1
1.3 Structure of the Report.....	1
2.Design Criteria and Specifications	1
2.1 Objectives	1
2.2 Scope.....	2
2.3 Success Criteria	2
2.4 System Constraints	3
3. Ethical, Social, and Environmental Considerations	3
3.1 Ethical Frameworks.....	3
3.2 Impacts on Safety, Fairness, and Accessibility.....	4
3.3 Social Impact and Societal Implications.....	4
3.4 Environmental Considerations	4
4. Design Methodology and System Architecture	5
4.1 Overall Architecture.....	5
4.2 Perception Subsystem	5
4.2.1 Lane Detection & Cross-Track Error Pipeline	5
4.2.2 Object Detection & Integration Pipeline	6
4.2.3 Perception Communication Pipeline	7
4.3 Planning & Localization Subsystem	7
4.4 Control Subsystem.....	8
4.4.1 Lane Keeping and PID	8
4.4.2 Stop Detection	9
4.4.3 Turn Execution	9
4.4.4 Line-Loss Recovery.....	10
4.4.5 Discrete State Machine.....	10

4.4.6 Obstacle/Duck Avoidance & E-Stop.....	10
4.4.7 I/O Commanding.....	11
4.5 Alternatives Considered & Tradeoffs	11
4.5.1 Perception.....	11
4.5.2 Planning & Localization.....	12
4.5.3 Control	12
5. Implementation and Integration	13
5.1 Subsystem Integration: Communication Between Subsystems.....	13
5.2 Integration Challenges Encountered	13
5.3 Development Timeline & Reaching Milestones	14
6. Verification, Validation, and Performance Evaluation	15
6.1 Perception Validation	15
6.2 Planning & Localization Validation	16
6.3 Control Validation	16
6.3.1 Lane Keeping and PID Performance	16
6.3.3 Turn Execution Validation.....	16
6.3.4 I/O Validation.....	17
6.3.5 Obstacle Detection Validation.....	17
6.4 Competition Results.....	17
7. Limitations, Risks, and Recommendations	18
7.1 Challenges, Limitations, and Assumptions	18
7.2 Risks and Lessons Learned.....	19
7.3 Societal and Environmental impacts	19
7.4 Recommendations for Future Iterations	19
8. Conclusion.....	19
References	21
Appendix A: Team Collaboration	22
Responsibility Distribution and Management	22
Workload Equity	22
Decision-Making and Conflict Resolution.....	22
What Worked Well	23
Appendix B: Individual Reflections	24
Rafael	24
Ishaan.....	24
Nolan.....	26

Declan	26
Appendix C: High Level System Overview	27
Appendix D: Planning and Localization.....	28
Appendix E: Perception	30
Appendix F: Control High Level Overview	36
Appendix G: List of Materials.....	36
Appendix H: MATLAB PID Tuner Image	37
Appendix I: Photos of the Picar	38

List of Figures

Figure 1: Flowchart depicting a high-level overview of the subsystems.....	27
Figure 2: Path Selection Flow Chart.....	28
Figure 3: Example Fare Routing	29
Figure 4: Perception Lane Detection & CTE Pipeline Flowchart.....	30
Figure 5: Perception Lane Detection & CTE Pipeline – CTE Branch Flowchart.....	31
Figure 6: Perception Lane Detection & CTE Pipeline – Stop-line Detection Branch Flowchart.	32
Figure 7: Perception Object Detection Model Training Pipeline	33
Figure 8: Object Detection Test Across All Vehicle Classes	34
Figure 9: CTE Implemented for Straight Vehicle State. Red curve displays trajectory, green dot indicates look ahead point where CTE is calculated.	35
Figure 10: CTE Implemented for Turns. Red curve displays trajectory; green dots indicate 10 look head points where CTE is calculated.	35
Figure 11: Flowchart depicting a high-level overview of the control subsystem.....	36
Figure 12: Screenshot of the MATLAB PID Tuner response curve.	37
Figure 13: Front facing picture of Picar.	38
Figure 14: Back facing picture of Picar.	39
Figure 15: Side facing picture of Picar.	40
Figure 16: Top facing picture of Picar.	41

List of Tables

Table 1: Breakdown on port allocations for TCP server.	13
Table 2: Example analysis results for validation.	16
Table 3: Breakdown of responsibilities per subsystem.	22
Table 4: Perception Object Detection Model Evaluation Results	34
Table 5: List of components used on the Picar.....	36

1. Introduction

1.1 Background

There is currently a huge change taking place in the transportation sector due to advances in driving automation, edge computing, and computer vision leading to the use of autonomous vehicles (AV) for ride sharing services. The Society of Automotive Engineers (SAE) International J3016 standard classifies autonomous systems across six levels, from Level 0, where the human performs all driving tasks, to Level 5, where the system operates without any human intervention [1]. Consumer vehicles today are at Level 2, managing steering and speed while requiring constant human supervision, while Level 4 autonomy (full self-driving within a defined area) remains limited to geofenced commercial robotaxi deployments such as Waymo's services in Phoenix and San Francisco [1] [2]. Data has shown that the autonomous ride hailing and last-mile delivery markets are projected to grow substantially in the next decade, driven by demand for cost-effective, continuous urban mobility [3].

AV deployment also carries significant societal implications. National Highway Traffic Safety Administration data suggests that 94% of serious crashes involve driver error [4], providing a compelling safety case for automation, though this figure has been criticized for overlooking contributing factors such as road design [5]. These benefits must be weighed against risks of labor displacement among professional drivers [6], and the possibility that increased vehicle convenience expands total miles traveled, potentially worsening urban congestion and emissions [7].

1.2 Problem Statement

While autonomous vehicle technology is advancing quickly at the industry level, building a reliable, fully integrated self-driving system on hardware with strict resource limits is still a major challenge. This project tackles that gap by developing an autonomous taxi prototype designed to safely handle a dynamic urban environment. The core problem is getting multiple complex subsystems, like computer vision, path planning, and real-time control to communicate and function together well. Each system has its own potential points of failure and timing limits, making integration complex. The goal is to engineer a system that does not just work under ideal conditions but can actively deal with issues and recover from failures.

1.3 Structure of the Report

The rest of the report follows the engineering lifecycle of the project. Sections 2 and 3 cover the core design requirements, constraints, and the ethical considerations that shaped the team's approach. Sections 4 and 5 break down the system architecture and explain how the team built and integrated the perception, planning, and control subsystems. Section 6 looks at how the vehicle performed during testing and validation. Finally, sections 7 and 8 wrap up the project's limitations, lessons learned, and recommendations for the future.

2. Design Criteria and Specifications

2.1 Objectives

The objective of the project is to create an autonomous taxi that can navigate the city of Quackston within the technical and operational requirements of the competition. These include being able to navigate the roadway in accordance with the road markings and signage, responding appropriately to pedestrians with

audio-visual signals, and completing pickup and drop-off through integration with the Vehicle Positioning and Fare System (VPFS). The passengers must be guaranteed safe transport with the help of the Cabin and Restraint Equipment (CARE) system, which safely secures the passengers during transport. In addition to abiding by these rules, the design must strike a balance between two performance goals: maximizing fare-based revenue and maximizing public reputation through safe, lawful, and predictable vehicle behaviour.

Not only must the design be capable of the route completion described above but it must be able to carry out those tasks in a dynamic shared environment. Quackston has intersections, one-way roads, stop and yield signs, crosswalks, pedestrians, other vehicles and construction zones which must be engineered around in the design process. The vehicle should also exhibit stable steering, reliable stopping, obstacle avoidance, emergency stop, and clear communication of vehicle intent through audio and/or visual indicators.

2.2 Scope

The scope of this project includes the design, integration and testing of this autonomous taxi built on the Picar-X platform in the context of the structured Quackston environment. There are three main subsystems: perception, localization/planning, and control, each of which carry responsibilities regarding the scope. The perception subsystem is responsible for detecting pedestrians and providing a visual basis to guide the turns made by control. Localization and planning are responsible for interaction with VPFS to select the optimal fare, create the optimal path for pickup and drop-off, and keeping track of map details at certain positions like stop signs, crosswalks, roundabouts, one-ways etc. Control is the culmination of the other two subsystems, as it combines the information that is provided into action. It is responsible for items like line following, braking, turning, signaling (audio-visual), stopping, and an indicator for the duck's presence in the vehicle.

Outside of the key responsibilities of the subsystems, there are competition-mandated requirements that fall within the scope as well. The vehicle must have an accessible emergency stop, rear turn signals, a rear brake or stoplight, an audio-visual alert system, a secure duck passenger restraint, and electrically safe and secure wiring. The implementation of these items must be mounted on the Duck Deck using nonpermanent methods, as invasive vehicle modifications are outside the scope of this project. The design does not attempt to pursue any complexities outside the standard competition needs; this means that the vehicle was not designed to complete the bonus challenges outlined in the course documents. The team has placed an emphasis on competition-critical requirements first and so, generalization of the vehicle's functionality outside of the specified Quackston map is out of scope. The design also relies heavily on the specifications for the identical ducks, the design has no capability to detect ducks that are not the uniform ones described in the course, this is outside the scope.

2.3 Success Criteria

At a system level, success means that the vehicle can navigate roads legally, stop when required, avoid collisions, respond appropriately to ducks and other obstacles, transport a duck passenger securely, and clearly communicate its intent through signals and alerts. Success also requires that the vehicle passes the pre-competition verification with no failures, attending the session and attempting operation. No severe violations and a positive reputational score at the end of competition is also a key indicator of success. Severe violations are failures that involve physical contact, result in loss of control, or present an immediate risk to people, ducks, buildings or equipment.

As for the technical success criteria, benchmarks that were outlined by control include a lane error within 2.0 cm, stopping precision of 0 to 5 cm before stop lines, and a deceleration time of no more than 0.5

seconds. Planning must reliably generate the shortest path and guide the vehicle correctly to the intended destination. Perception must detect ducks with a minimum mean average recall of 0.85 and a minimum mean average precision of 0.75 and convey this information to control so the vehicle can respond safely. The CARE system will be considered successful if the duck remains secure through sudden stops, turns or collisions, and if it has a passenger detection mechanism.

2.4 System Constraints

The car is constrained by the base Picar-x platform as the team is not permitted to make any invasive changes to the base vehicle. All mounting and building must be done using the Duck Deck and be removable while also passing the vehicle dimension constraints: 300mm height, 170mm width, and 300mm length. The team must also work within the bounds of the hardware provided, which can have a large effect when it comes to a complex integrated project with image processing, control and planning. The coral TPU that is used to offload the object detection has an SRAM cache size of 8MB which places an upper limit on the detection model's size. Due to the original wiring of the Picar, there are only two remaining digital pins which can place a limitation on the potential for added hardware.

The design is limited to a budget of \$50 CAD; this can place a limitation on the number of added components and will force the team to design around low-cost additions that are mission critical. The design is also constrained through an ethical lens by the importance that safety takes over speed, profit or competition score. Socially, the vehicle should operate predictably and provide clear signaling, lawful lane discipline, reliable stopping and behaviour that preserves public trust. This is important as the vehicle will be operating on a shared road and should not endanger or create difficulties for others in the process.

3. Ethical, Social, and Environmental Considerations

3.1 Ethical Frameworks

Used in Design The ethical decision-making taken by the team can be described most accurately as a safety-first framework. This framework is supported by the precautionary principle and consequentialist harm minimization. The precautionary principle is an ethical policy which at its root states that if something may cause major damage, the actors should not wait until the damage is fully proven before taking precautions [8]. This is a principle that the team chose to follow as the lives of pedestrians and passengers are at stake in the competition and that the design should err on the side of caution. Consequentialism is the view that decisions should be judged by their outcomes [9]. In the context of the competition, it means that design choices should be evaluated based on their ability to reduce or prevent harmful outcomes like 4 collisions, unstable motion, or passenger ejection. Despite the application of consequentialism in some regards, pure consequentialism was not applied in isolation. In a pure consequentialist lens, there is an argument to be made that the vehicle should be evaluated only on competition outcome at any cost. Instead, consequentialism was used in the lens of harm minimization, so that safety obligations were prioritized over things like speed, fare completion or score. This remains consistent with the team's design ethos mentioned in the project proposal of safety-first, evidence-based decisions, and reliable simplicity.

The framework mentioned above informed engineering trade-offs that the team took. The design prefers simpler solutions that meet the system requirements first rather than an unreliable overly ambitious one. The position that the team took when it came to uncertainty was also shaped through an ethical lens. If the system is uncertain there are safe actions to take like slowing and stopping rather than exhibiting

more aggressive behaviors that may result in a higher competition score at the detriment of the passenger or others.

3.2 Impacts on Safety, Fairness, and Accessibility

Safety has been the common theme when it comes to ethical and engineering concerns as the vehicle operates autonomously and transports a passenger. This theme has shaped the design from the beginning and has strong alignments with the specifications outlined by the course requirements. The course requires that the vehicle operates safely with controllable motion, reliable braking, signaling, emergency stop capability, and secure hardware integration. Passenger safety is especially important as their lives and the lives of others are the most valuable under this framework. It is because of this importance that the CARE design was implemented with a secure elastic band seat belt binding the torso of the duck. Although this design may not have been the most comfortable for the duck, the primary objective of the team is its safety, which this design provided.

Under the framework that the design was built under, the team favors reliable simplicity overall and once a requirement is fulfilled, resources should be placed towards ensuring reliability of other systems. Ideally, the CARE system could have been optimized for comfort, however, the team decided it was best to ensure that other requirements to ensure the safety of others took first priority. Fairness in this project is addressed by refusing to shift risk onto others in exchange for a better performance. The team's philosophy rejects the idea of aggressive maneuvers to gain a higher score at the detriment of other teams. In this project the team operated in a lens that would assign the burden of uncertainty to the vehicle itself rather than pedestrians, passengers or surrounding taxis through safe operation.

Accessibility was addressed for the surrounding environment through audio-visual alerts that are used to communicate with other cars as well as pedestrians. As for loading and unloading of the passenger there were no accessibility measures that were taken, this is because the scope of the project does not require it. All ducks are uniform, none have broken limbs or disability issues, and ducks should be placed securely in their cabin by a handler. Ultimately, for loading and unloading, the team operates under an "accommodate one, accommodate all" framework under the specified scope of the project.

3.3 Social Impact and Societal Implications

The impact that societal implications play on engineering stems from the idea that autonomous systems are judged not only by their function, but by whether they can be trusted. In the vehicles design, trust is manifested through safe and predictable behavior. This reflects that technical capability alone is insufficient if the vehicle does not behave in a way that is socially acceptable. This predictable behavior was engineered into the design by maintaining reliable communication with others on the road through the signaling. The team also purposefully lowered the speed of the car to increase reliability of turns, to give the Pi and Coral more processing time, and to increase the smoothness of the ride for the passengers. All of the engineering decisions mentioned culminate in a more trustworthy and predictable design in the lens of society.

3.4 Environmental Considerations

Although environmental considerations played a smaller role than safety and social impact, it was still relevant to the design process. Some environmental considerations are inherently built into the course with the specifications regarding invasive modifications to the car, and removable mounting on the Duck Deck. These rules encourage reuse of the base system and discourage modifications that may render the car unusable for a future group. In addition to the inherent environmental considerations of the course, the team's preference for reliable simplicity also has environmental value. The philosophy of the team is

to have a simpler design targeted at completing the minimum requirements to successfully complete the objective. This meant that the design would require fewer components and therefore produce less waste, this is also emphasized by the courses tight budget constraint which further reinforce reliable simplicity and its importance in the design of this vehicle.

4. Design Methodology and System Architecture

4.1 Overall Architecture

The autonomous taxi system was designed around three subsystems: Perception, Planning and Localization, and Control. Each run as an independent process on the Raspberry Pi 5. Communication between subsystems was handled through a TCP server. A high-level diagram of the subsystem interactions is shown in Figure 1.

The Perception subsystem is responsible for processing data from the camera and publishing information to the other subsystems. It runs on a Coral USB Accelerator (TPU), which offloads object detection inference from the Raspberry Pi's CPU, enabling real-time detection. Perception publishes lane trajectory data (cross tracked error and lookahead distance), distance to the next stop line, and duck detection events.

The Planning and Localization subsystem maintains a directed graph representation of the Quackston map and uses Dijkstra's algorithm to compute optimal routes between the vehicle's current position and fare pickup or drop-off locations. Vehicle position is obtained from the competition's Vehicle Positioning and Fare System (VPFS) API. The planning subsystem translates computed paths into a sequence of direction commands and publishes them to the Control subsystem.

The Control subsystem is the lowest-level process and is responsible for all real-time vehicle actuation. It subscribes to direction sequences from Planning and trajectory data from Perception, and executes lane following, intersection handling, turning, and safety behaviors. Control publishes its current mission state back to Perception so that Perception can adapt its processing mode; for example, switching to stop-line scanning when the *no_line* flag is active, or tilting the camera down before a trajectory turn.

The full hardware list is summarized in Table 5.

4.2 Perception Subsystem

The perception subsystem is composed of three primary pipelines: lane detection & cross-track error, object detection & integration, and perception communication. Each pipeline was designed with preprocessing, multistep checks, and efficient frame handling to minimize false positives and communication latency.

4.2.1 Lane Detection & Cross-Track Error Pipeline

The lane detection module, *lane_detection_and_cte.py*, serves as the primary perception output for autonomous lateral control. It accepts a raw BGR camera frame and returns either cross-track error (CTE) for trajectory during turns or a distance measurement to an upcoming stop line, depending on the current vehicle state. All processing is performed on a bird's eye view (BEV) projection of a green segmentation mask for geometric correction and spatial accuracy.

Each frame passes through a common preprocessing stage regardless of operating mode. The input frame is first converted from BGR to HSV colour space, and a binary mask of the green lane tape is extracted using calibrated HSV bounds. Morphological opening and closing operations are then applied to suppress noise and fill gaps in the mask. Then, the module queries the ZMQ server for vehicle state, which returns a state (ST, R, L1, or L2) and a *no_line* flag, which determines which of the two processing branches are executed. Figure 4 illustrates this common preprocessing stage.

4.2.1.1 Stop-Line Detection Branch

When *no_line* is *True*, the module searches for a horizontal line crossing the vehicle's path rather than computing CTE. A straight-perspective BEV warp is applied during a stop-line approach. The lane center x is determined from the bottom 20% of the BEV, and a vertical corridor of ± 54 pixels around this lane center is excluded to prevent the center green tape from being included in the search. Then, the module looks for pixel arms on either side of this vertical exclusion zone and uses row histograms for each arm to find the peak candidate row, that is, the row with the highest pixel frequency.

Next, five conditions must be passed before a detection is accepted. 1) The peak rows of the left and right arms must be within 100 pixels of each other vertically. 2) The combined pixel count across both arms must meet the minimum threshold. 3) The pixel span in the peak row must exceed a minimum width. 4) & 5) A line fitted to the candidate band must satisfy maximum slope and mean residual constraints. Altogether, these checks reject curved, noisy, or near-vertical structures, minimizing false positive detections. If all checks pass, the distance to the stop line is determined as the pixel distance from the car to the peak row, scaled by the meters-per-pixel factor. This branch returns this distance alongside a null CTE output. Figure 6 illustrates the stop-line detection branch.

4.2.1.2 CTE Trajectory Branch

When *no_line* is *False*, the module computes CTE for lateral trajectory control during turns. A perspective warp is applied using either the straight or turn BEV matrix depending on vehicle state and a region-of-interest (ROI) mask is applied to isolate the lane pixels.

The ROI masking implementation differs between straight and turning states. For straight driving, a simple vertical strip of ± 54 pixels around the lane center is used. For turning, the module first scans downwards from the midpoint of the image to find the intersection row, which is the first row where the pixel span exceeds three times the ROI half-width, which indicates that the lane has opened into an intersection. Below this row, the mask is extended laterally in the direction of the turn to capture the curving lane geometry, and a second filter removes detections in the opposing region.

After masking, a cubic polynomial is fit to the filtered pixel coordinates, expressing the lane's x position as a function of the BEV row index (y coordinates). Before sampling, the module checks if the detected pixels span a sufficient vertical range, ensuring that subsequent intersections are not included and that the topmost pixel is not too close to the car since this would lead to unreliable trajectory based on experimental testing. Next, ten evenly spaced reference points are sampled along the polynomial from near the car to the lookahead horizon. At each reference point, the CTE is computed as the lateral offset between the lane position the vehicle center. These ten CTE values and lookahead distances are primary outputs of this branch, as displayed in Figure 5. The implementation of CTE on straights and turns is shown in Figure 9 and Figure 10 respectively.

4.2.2 Object Detection & Integration Pipeline

Object detection was used solely for the purpose of duck detection. 300 duck images were used to train the duck model, a third of which were collected on the road in exact scenarios where ducks would regularly be seen, and the rest were taken with differing clear backgrounds at varying distances and poses.

Pascal VOC format was used for the annotation and training was done in the provided Google Colab file where an EfficientDet-based model was created. The full training pipeline can be seen in Figure 7.

Although the model was strong when it came to detecting ducks, there were still many false positives. One problem came from the wide field view of the camera. For instance, a duck in the opposite lane could still be detected despite it not being in the cars' direct driving path. This was avoided by creating a trapezoid mask whose bounds were hand tuned so that the region matched the visible shape of the lane.

Another false positive case came from ducks that were off-road but still in sight, these were ducks placed on islands surrounded by white lane boundaries. In these cases, the car was not going to drive into the ducks but would still stop and honk. This case was avoided by clipping the trapezoid lane mask above a detected forward white boundary.

Additional filters were added to avoid detections that were clearly unreasonable. A bounding-box area gate was used so that only areas between 4000 and 10000 pixels were processed. In addition to this area gate, a colour gate was applied to each bounding box that was detected. The detection was only accepted if the ratio of yellow pixels inside the box exceeded 0.4. Detections would have to pass through both filters to make it to control, this greatly reduced false negatives and made for a reliable duck detection system. Detections were passed to control in a tuple of object type and distance, although ducks were the only detected objects.

4.2.3 Perception Communication Pipeline

The *perception_server_comms.py* module manages all communication between the perception subsystem and the mission state server. It uses ZMQ, with subscriber (SUB) and publisher (PUB) sockets initialised once at load time and reused across every frame. This persistent connection design avoids overhead of repeated TCP handshakes, thus minimizing frame communication latency.

A SUB socket connects to port 5555 and listens for *MISSION_STATE* messages, from which it extracts the vehicle state and *no_line* flag, which are stored in a *_VehicleStateManager* instance and updated non-blocking on each call to *get_vehicle_state()*, ensuring that socket never blocks the perception pipeline. A PUB socket connects to port 5556 and is used to push perception data to the server under a *_PerceptionPublisher* instance. The publisher sends CTE, lookahead distance, and distance to line values to the *TRAJECTORY* topic, timestamped at send time. Duck detections are published under the *DUCK* topic, with both the forward distance and horizontal offset from the vehicle.

The use of separate SUB and PUB modules which are initialised at load time, as well as non-blocking protocols, makes this architecture suitable for the frame-by-frame perception loop where reducing communication latency is a high priority.

4.3 Planning & Localization Subsystem

The planning and localization subsystem was responsible for selecting the optimal fare for the taxi to select, as well as delivering a series of directions to the control subsystem to direct the taxi through the selected fare.

A prerequisite for accurately determining the optimal fare to select was knowing the car's relative position on the map and the start/end position of each fare to be chosen from. To do so, the team used a graph of nodes and edges to simulate the map of Quackston. The initial set of nodes was placed at intersections around the map, with edges between them representing roads. The graph was implemented as three lists: one of x coordinates, one of y coordinates, and the adjacency list, with indices representing nodes. The team then added supplementary nodes for missing intersections, crosswalks, and surrounding the roundabout.

With the graph in place, the team implemented a version of Dijkstra's Shortest Path Algorithm to determine the shortest path between any two points. The algorithm included the capacity to add and remove temporary nodes to the graph to represent start and end positions, important for precisely routing the car between its position and a destination.

The car's position and important information about the available fares were gathered via the VPFS API provided by the teaching team, to be passed as necessary to the subsystem's functions.

For each available fare, the pathing code ran the Dijkstra's algorithm once between the car's starting position and the pickup point, and then from the pickup point to the drop off. Then, the overall relative value of the fare was computed using Equation 1.

After the score for each fare was calculated, the path between the start and end points was translated from a list of nodes to a list of directions that the control subsystem could input into the car. The classification of directions worked by comparing the vector between the cars' present position and its next node to the vector between the next node and the node following it. Many of the combinations of nodes were hard coded because the team found during live track testing that the car often made mistakes during turns when directions were limited to left, right and straight. To accommodate the car, the lateral directions were further classified into three types of left and two types of rights. The regular turn classification became a default fall back for combinations of nodes that were not included in the manually classified list. Finally, the team included the directions RAEN and RAEX for when the car entered and exited the roundabout.

Once a fare had been selected and its directions had been calculated, that information was passed from Planning to Control via a Zero Message Queue (ZMQ) WebSocket. The pathing code consistently interacted with the VPFS API to get the next fare, send directions to Control, continuously update directions thought out the course of the fare (in case a mistake occurred or the car took a wrong turn) and repeat through a round of the competition.

A plotted example of the graph with the car represented as a red dot, the passenger as a green dot, the destination as a yellow dot, and the paths to the passenger and to the destination represented in purple and pink respectively can be found in Figure 2. By competition the software was able to consistently plot directions from any start and end position just like in the given example.

4.4 Control Subsystem

4.4.1 Lane Keeping and PID

The primary design choice for the control system was to use a PID controller for lane keeping, guided by the grayscale sensor. The grayscale sensor proved fast and reliable in controlled environments, making it the preferred input over the camera for real-time lane-keeping feedback. The PID controller computes a steering correction from the lateral error between the detected lane position and the car's center. The equation used by the controller is shown below:

$$s_i = \text{clip} \left(\frac{(r_i - o_i) - \text{cal}_{\min i}}{\text{cal}_{\max i} - \text{cal}_{\min i}}, 0, 1 \right)$$

where r_i is the raw ADC reading, o_i is the per-sensor hardware offset, and $\text{cal}_{\min i}$, $\text{cal}_{\max i}$ are the calibrated floor and ceiling values for sensor $i \in \{Left, Center, Right\}$.

The lateral error is then computed as a weighted average position of the line across the sensor array:

$$e = \frac{S_L - S_R}{S_L + S_C + S_R} \cdot 100$$

where a positive error indicates the line is to the left of center (steer left) and a negative error indicates the line is to the right (steer right). The denominator normalizes the total signal strength to make the error tolerant to variations in line contrast. When the total signal falls below a threshold (i.e., no line detected), the last known error is returned amplified by a factor of 1.3 to encourage corrective steering:

$$e_{lost} = 1.3 \cdot e_{last}$$

To determine the controller gains, the integral term K_i was set to zero, as line-following systems of this type are susceptible to integral windup (the accumulation of residual error over time that causes the controller to over-correct) [10]. For the proportional and derivative terms, MATLAB's PID Tuner was used to produce an initial estimate from a modeled impulse response of the steering system (shown in Figure 5). This produced initial values of $K_p = 0.18$ and $K_d = 0.03$, which were then fine-tuned empirically on the track to account for real-world nonidealities such as surface friction and sensor noise. A deadband of 0.5 was introduced to suppress small errors below a meaningful correction threshold, and a moving average error buffer of length 2 was applied to smooth noisy sensor readings before they entered the controller. The PID control equation (with $K_i = 0$) is:

$$u[k] = K_p \cdot e[k] + K_d \cdot \frac{e[k] - e[k-1]}{\Delta t}$$

The output is then clamped to the servo limit:

$$\delta[k] = \text{clip}(u[k], -\delta_{max}, \delta_{max})$$

Where $\delta_{max} = 30^\circ$.

4.4.2 Stop Detection

To detect stop and turn lines, the grayscale sensor was monitored for the condition where all three sensors activate at the same time, indicating a line spanning the full width of the sensor array. This occurs reliably at intersections and stop lines in Quackston. However, because this condition also triggers at crosswalks and general intersections, the mission state machine was designed to contextually interpret the event. This meant it was either stopping, turning, or passing straight through depending on the current state.

4.4.3 Turn Execution

For turn execution, the primary mode used a camera-guided trajectory approach. Upon reaching a stop line, the vehicle halts and tilts the camera downward to capture a bird's-eye-view of the exit lane. The perception subsystem then sends a list of (distance, CTE) trajectory points to the control system over the server. The control system then applies a bicycle model approximation to determine the required steering angle at each step along the list. Given a lookahead distance L (in cm) and a lateral offset (CTE) y at that distance, the required steering angle is:

$$\theta_{turn} = \tan^{-1} \left(\frac{2 \cdot W \cdot y}{L^2} \right)$$

where $W = 9.5$ cm is the measured wheelbase of the PiCar-X.

Because the raw bicycle model was seen to consistently understeer, an angle offset of 27 degrees was added to tighten the response. Also, differential motor PWM was applied throughout the turn, increasing the outer wheel's speed relative to the inner wheel to minimize slip which was an issue identified after the PiCar-X began sliding at uniform speed. After the timed turn completes, the grayscale sensor scans for line reacquisition. A minimum timer was imposed before grayscale could end the turn, preventing the car from locking onto an adjacent lane line before completing the maneuver.

For turns without a stop line (such as right-angle exits without a preceding crossline) an outside-wheel-only turn was used. In this mode, the outer wheel drives forward while the inner wheel is driven in reverse, executing a pivot. The grayscale sensor again signals when the exit lane is acquired. A fallback blind turn was also implemented for cases where the camera could not produce a valid trajectory, applying a fixed maximum steering angle for a predetermined duration before falling back to grayscale scanning.

4.4.4 Line-Loss Recovery

A five-state line-loss recovery system was implemented to handle situations where the lane disappears unexpectedly. The system records a rolling history of recent motion samples including speed, steering angle, and line-seen status. If the line is lost for longer than a certain timeout (1.0 s), the system retraces the vehicle's path in reverse (replaying recorded motion steps in reverse order) until the line is re-acquired for two consecutive frames, at which point normal lane following resumes. A cooldown prevents repeated rapid recovery loops.

4.4.5 Discrete State Machine

The higher-level mission logic was implemented as a discrete state machine with the following states: STRAIGHT, APPROACH_STOP, LEFT_1, LEFT_2, RIGHT, ROUNDABOUT_ENTRY, ROUNDABOUT_CIRCULATE, ROUNDABOUT_EXIT, CROSSWALK, CALIBRATE, and IDLE. LEFT_1 and LEFT_2 distinguish between intersections requiring one versus two stop-line crossings before the turn. A no_line flag was added to each turn state for scenarios where no stop line is present, triggering the outside-wheel turn mode instead. The ROUNDABOUT states handle roundabout entry and exit using the same outside-wheel turn geometry, but with different line-skip counts than standard turns. The state machine receives direction sequences from the planning subsystem over ZMQ and advances through states as intersection events are detected. A high-level overview flowchart is shown in Figure 11.

4.4.6 Obstacle/Duck Avoidance & E-Stop

For obstacle avoidance, the ultrasonic sensor readings were median filtered over a buffer of three samples to reject spike readings. If the filtered distance fell below a 10 cm threshold, the vehicle stopped immediately, sounded the horn, and activated hazard lighting until the obstacle cleared. Duck detections from the perception subsystem were received as a DUCK topic ZMQ message. Upon receipt, the current mission state was saved, the vehicle stopped, hazard lights and horn were activated, and the system waited until the duck flag expired before resuming. The emergency stop (e-stop) was implemented as a keyboard interrupt that froze all motor commands, saved the current state, and blocked the control loop until the resume command was received, at which point the saved state was restored exactly once.

4.4.7 I/O Commanding

IO integration included left and right turn signals driven via PWM blinker LEDs, brake lights activated when speed fell below 1.0 PWM, a horn triggered via the Music module for duck and obstacle events, and a limit switch on GPIO pin 4 used to signal passenger boarding confirmation after a stop. The all-lights hazard mode (blinking both blinkers and brake light simultaneously) was used for duck stops, obstacle stops, and e-stop events.

4.5 Alternatives Considered & Tradeoffs

4.5.1 Perception

4.5.1.1 Object Detection Scope Revision

Initially, the team had outlined object detection scope to include the following object classes and their respective sizes: stop sign (50), do not enter sign (50), duck (100), one-way (100), yield (50), and vehicle (50). After training the model on this dataset, consistent false positives and negatives were seen with the one-way, vehicle, and duck classes. To address this, further images and annotations were completed, however, the issues with the vehicle and duck classes persisted. Upon running the object detection model during vehicle operation, the team noticed a significant reduction in performance and accuracy across all object classes due to the frame lag and poor resolution. Consequently, it was decided to consider modifying the object detection scope to prioritize training allocation to mission critical classes.

To begin, the vehicle class was removed from the dataset due to its consistent false positives in detecting the black features of the road as vehicle tires, as well as the cameras inability to clearly capture other vehicles while in motion. To ensure safety, proposed changes were discussed with the control sub-team, after which, rigorous testing of the ultrasonic sensor was performed to ensure that it detected other vehicles timely and with enough margin to ensure that the team's vehicle stopped safely, ensuring the safety of passengers and other Quackston citizens. Due to the control subsystems use of the grayscale line following sensor to detect stop lines, and the incorporation of all intersection markings in the planning map, all sign classes were removed from object detection scope since this application was redundant.

By removing these classes through careful safety and subsystem integration considerations, the perception team narrowed the object detection scope down to purely the duck class. This was the most critical object detection class since the ultrasonic sensor could not consistently detect the ducks soon enough, or at all, if they were laterally offset from it. Thus, to prioritize the safety of Quackston citizens, this class size was expanded to 300 annotated images, with over half of these being collected on the road in the exact conditions. Making this tradeoff in object detection scope proved to be successful since the model now consistently detected ducks in various locations, distances, and orientations during vehicle operation, ensuring no missed detections.

4.5.1.2 Lane Detection

The original lane detection implementation proposed utilization of the object detection model to detect the outer white lane boundary, where the vehicle would aim to center itself a specified range to the left of the right white lane boundary. However, after the green center tape was added, the team decided that lane detection would focus on the green center tape instead since knowing the distance, or offset, from this line would be the most effective and straightforward for integration with control. Next, the team considered an alternative to utilizing the object detection model because research indicated that from the camera's perspective, the line would appear distorted and curved and that any depth perception would face real world scale inaccuracy. Furthermore, bounding boxes would limit the preprocessing and image processing techniques that could be applied. Upon conducting detailed research and planning, the lane detection & CTE pipeline was created, which not only enabled more accurate and custom lane

masking and filtering, but also tackled the distortion and scale inaccuracies using BEV transform. Ultimately, this alternative resulted in the calculation of CTE at various reference lookahead distances to keep the vehicle centered in the lane and assist with turns.

4.5.2 Planning & Localization

A variety of system design choices were made through the course of development of the Planning and Localization subsystem. The team had to decide how directions would be calculated and then how they would be transmitted to Control. There were not necessarily wrong choices for how these decisions would be made, but each option had costs and benefits that had to be considered, especially given the context that we wanted to make integration with other subsystems as easy and seamless as possible.

There were several options for the implementation of the graph representing the map of Quackston. Initially, a class-based Java influenced style was considered. However, because of the speed and space demands of instantiating many class objects and the need for constant refactoring of code referencing the class whenever a change needed to be made, the team decided to use a more C style graph implementation, with several lists of different variables making up the graph, and individual nodes being accessed by their index. This approach made it easy to add additional information to the graph as necessary (in case we later wanted to add additional information about specific nodes).

The team also deliberated between creating a dense and sparse graph. A denser graph would have improved the precision of the calculated length of each route and given us the option to implement directions via a way point system, where the heading of the car was compared with the relative position of the next way point to gradually turn the car. However, because of the importance of recalculating directions to make up for missed turns or mistakes, an extremely dense graph would have dramatically increased the running time for the pathing algorithm. Furthermore, this approach would have required a technical programmatic approach to automatically place way points throughout the map of Quackston, or having the team painstakingly plot out hundreds of points across the map. For these reasons, the team turned to a sparser graph implementation, only placing nodes at key turn making decision points.

Another key decision was which pathing algorithm would be chosen to navigate the car. Because the graph of the map has exclusively non-negative edge weights, the two obvious options were Dijkstra's Shortest Path Algorithm and the A* algorithm. Though both algorithms have the same worst case time complexity, A* is generally preferred for mapping because its directed nature frequently means that it finds the path to the destination more quickly than Dijkstra. However, given the previous decision to keep the graph relatively sparse, and the increased technical difficulty of implementing the A* algorithm, the team decided that doing so would be unnecessary over engineering. Instead, the team developed a version of Dijkstra's algorithm, which correctly and quickly calculated the optimal paths for the car.

After the above decisions had been made, the team considered how turns would be communicated from Planning to Control. At first, we considered importing the Planning files within the Control main loop and simply running methods as necessary, but in order to cleanly delineate responsibilities and facilitate the concurrent development of both systems, the team decided to run a separate file to determine directions and ship the outputs to Control via a ZMQ web socket port.

4.5.3 Control

The original lane-keeping design proposed a cascaded PID controller blending both grayscale and camera feedback. The camera would provide a longer-range CTE estimate for feedforward steering, while the grayscale would provide high-frequency near-field correction. This was abandoned due to time constraints and the sufficient performance of the grayscale-only system at competition speeds.

For stop-line detection on no-line turns, the camera was initially intended to measure the distance to the approaching lane line, triggering the turn at a configurable proximity threshold. This was rendered unnecessary when additional line markers were placed on the track before the competition, allowing the grayscale full-cross detection to work reliably for these turns. At that stage, the grayscale trigger was more reliable than the camera distance estimate under variable lighting.

The choice of PID control over a Kalman filter was made based on teaching team guidance and the relative implementation simplicity of PID. In retrospect, a Kalman filter fusing grayscale error and camera CTE could have improved tracking stability at higher speeds, where the PID controller showed visible oscillation. This is noted as a future improvement.

The decision to use a distributed ZMQ architecture rather than a single codebase was driven by team structure and maintainability. With the control module alone exceeding 1000 lines, merging perception and planning into a single process would have significantly increased integration complexity and the risk of cross-team regression. Running the ZMQ broker locally on the Raspberry Pi kept inter-process latency well under one millisecond, making the overhead negligible relative to the 10 ms control loop period.

5. Implementation and Integration

5.1 Subsystem Integration: Communication Between Subsystems

All subsystem-subsystem communication occurs over loopback TCP using ZMQ PUB/SUB sockets, with the key port assignments shown in Table 1. Since all processes ran on the same device, round-trip message latency was consistently under one millisecond. This architecture allowed each team to develop and test their subsystem independently while sharing a well-defined message interface, reducing integration risk significantly. Table 1 displays the topic communication method between subsystems.

Table 1: Breakdown on port allocations for TCP server.

Port	Direction	Topics
5555	Control → Perception	MISSION_STATE, TELEMETRY
5556	Perception → Control	TRAJECTORY, DUCK, DISTANCE_TO_LINE
5557	Planning → Control	DIRECTIONS, STOP

Planning and localization fed directions to control via a ZMQ socket. Directions for the car to its next destination were consistently delivered through the course of each fare to ensure the car stayed on course and corrected for any incorrect turns. Planning also delivered a software-based stop instruction whenever the car came within 20 cm of the pickup or drop-off location until the VPFS API registered the step in the fare had been completed. Directions were communicated as a JSON object over the web socket. As discussed in 4.2.3 Perception Communication Pipeline, the perception team fed CTE, look ahead distances, distance to line values, and duck detections to the server.

5.2 Integration Challenges Encountered

A major integration challenge was determining how to communicate between the three primary subsystems, since each subsystem's modules required independent execution but also depended on outputs from other subsystems. Although UDP was the recommended protocol since it ensured fast speeds, the team decided to use TCP (as outlined above) for its error checking capabilities and ordered package messaging. An issue the perception team ran into when integrating their outputs with the server

was communication latency. Initially, the code was developed such that on each function call, the SUB and PUB sockets would be initialized, which occurred on each frame. As a result, this created significant frame latency in the overall perception pipeline, causing delays in CTE computation and object detection. To resolve this issue, the SUB and PUB sockets were initialized once at load time and reused across every frame. This persistent connection design avoids overhead of repeated TCP handshakes, thus minimizing frame communication latency. After the perception and control teams debugged this issue, this integration strategy was standardized across all sub-teams.

A critical issue occurred when syncing planning and control in which the direction queue would become out of sync between the control code's main loop and the separately running planning subsystem code. Especially in the case where directions were recalculated whilst a turn was being completed by the car, the systems were prone to falling out of alignment. To fix this issue, direction instructions were sent with time stamps. When they arrived at control, the main loop checked if the newly arrived instructions came in before or after completion of the most recent turn. If they came in after, they were ignored as stale. This ensured that only directions calculated while the car was travelling along a straight were used by control, ensuring harmony between the two systems.

Another issue that the team encountered when integrating planning and control was a misunderstanding of how the nodes in the graph were designed with respect to control's needs. As discussed above, the control system relies on the grayscale sensor detecting stop/intersection lines to know where to stop and which line to turn on. However, from the planning system's perspective, when a node was added in an intersection, it was not considered that the grayscale sensor would see the stop line and two subsequent green lines as the vehicle travelled through the intersection. Consequently, when the team tested the autonomous navigation, the control code only received one straight command for going through an intersection, when it was expecting a stop command for the stop line and two straight commands to pass over the intersecting green lines. To address this gap, special cases were added to the planning system for when the direction of travel was straight through an intersection, which added one or two extra straight commands to the queue as needed. Furthermore, all the stop lines were also added such that the queue would contain stop commands as well. When creating these solutions, the team noticed a communication gap in how the roundabout nodes were treated by both systems, and these were corrected. Overall, these issues highlight the importance of clear communication and systems integration planning at the beginning of a project to ensure that the software understands how the physical hardware is being implemented and how it is using software outputs.

5.3 Development Timeline & Reaching Milestones

The team has implemented project scheduling using GitHub as its primary project management platform. At the beginning of the project, the team created a Kanban and Gantt-style project timeline with key milestones to help brainstorm and outline what tasks needed to be done and understand subsystem dependencies. Since then, the Kanban has been used to monitor pending, in progress, complete, and backlogged tasks. The Gantt-style project timeline served as a reference to ensure each sub-team was on track in respect to its large milestones and critical dates. Within each sub-team, it was the lead's responsibility to ensure they were on track with their tasks, and when delays arose, they informed dependent teams of their status and revised projections. Meeting twice weekly helped ensure all teams were aware of delays, which enabled efficient use of time regarding pivoting tasks when needed.

The largest and most critical Gantt chart milestone that was consistently referred to throughout project development was the beginning of subsystem integration, which was scheduled to begin the week after reading week on February 24th. This was a crucial milestone because this would have given the overall

team just over three weeks to integrate, refine, and test their subsystems together. This was significantly important for perception and control integration since autonomous movement heavily depended on the two systems working seamlessly with one another. However, due to delays from both sides, primarily due to object detection model data gathering, the need for repetitive retraining, and recalibration, this milestone target was not met. Instead, these two teams began integration on March 3rd, one week later than projected. In retrospect, this delay did not significantly affect performance and the system solution since from here on out, the two teams utilized all available testing time twice a week to ensure CTE, distance to line, and object detection integration occurred smoothly. In and of itself, this presented a challenge with regards to scheduling and prioritizing testing since there was only one car and two teams that needed to test and debug code. However, the two teams were able to work efficiently by utilizing downtime to complete pending tasks or by for example, the perception team assisting control with calibration and testing.

The integration of localization and planning with control was more significantly delayed due to the inability to test VPS early on and miscommunication of requirements as discussed above. As a result, most of this integration occurred during the weekend before the competition, when the mat was laid out in the foyer. This rushed integration caused logic errors and bugs in the integration code, which took long to identify and address, taking time away from full system integration and autonomous fare testing. In hindsight, if this integration had started earlier, or system requirements were better laid out, this could have been avoided. Therefore, the team's experience illustrates how unprecedented challenges, or in this case, requirements, can significantly alter projected timelines and affect future testing plans.

6. Verification, Validation, and Performance Evaluation

6.1 Perception Validation

To verify and validate the duck detection model, the team evaluated its performance using standard object detection metrics, primarily mean average precision (mAP) and mean average recall (mAR) across multiple intersection over union (IoU) thresholds. These metrics were selected based on their industry use in benchmarking detection systems and their ability to capture both localization and classification performance. Predefined design criteria required a minimum mAP of 0.75 and a minimum mAR of 0.85, with particular emphasis on performance across IoU thresholds of 0.5 to 0.95. Table 4 summarizes the evaluation results.

The model achieved an overall mAP of 0.797, exceeding the minimum requirement of 0.75. Performance at lower IoU thresholds was exceptionally strong, with AP@0.5 having a perfect score of 1.0, indicating that the model consistently detects ducks with high confidence when the localization constraints are relaxed. At stricter thresholds, such as AP@0.75 = 0.971, the model still demonstrates strong localization capability, indicating that bounding box predictions are tight and accurate. When looking across object scales, performance remained consistent, with AP for medium objects at 0.813 and large objects at 0.770.

In regard to recall, the model achieved a mAR, approximated using AR_{max100}, of 0.87, which passes the required threshold of 0.85. This result indicates that the model can detect the majority of ducks present when allowed sufficient number of detections per image. However, recall at lower detection limits, like AR_{max1} = 0.713, is significantly lower, which suggests that the model may miss detections when constrained to fewer predictions. Recall performance across medium (0.882) and large (0.843) objects is strong, thus reinforcing the model's reliability in typical operating conditions.

Overall, the model meets and exceeds the defined design criteria for both precision and recall. The high AP and AR values across multiple IoU thresholds validate that the model is accurate and robust in

detecting ducks. Experimental testing confirms this, as all ducks were detected and avoided, resulting in no pedestrian hits during the pre-competition safety verification and actual competition.

6.2 Planning & Localization Validation

The Planning and Localization subsystem's testing and validation consisted of two parts. Firstly, the code was stress tested with a variety of different simulated start and end locations for fares, ensuring that it consistently output accurate directions. Secondly, the team conducted track testing to ensure that these directions were correctly transmitted to the Control subsystem. By the end of the iterative testing, the directions were consistently and correctly delivered to the car.

6.3 Control Validation

6.3.1 Lane Keeping and PID Performance

Lane-keeping performance was validated through structured logging of the control loop. At each iteration, the time elapsed, mission state, smoothed lateral error, steering angle, and vehicle speed were recorded to a CSV file. These logs were then analyzed using a purpose-built script (`analyze_logs.py`) that computed key statistics including mean absolute error, median absolute error, the percentage of samples with large errors ($|\text{error}| \geq 70$), steering saturation percentage (steering angle $\geq 24.9^\circ$), and a per-state breakdown of these metrics. A representative summary of results from a nominal straight-line run is shown in Table 2 below.

Table 2: Example analysis results for validation.

Metric	Value
<i>Mean error </i>	23.91
<i>Median error </i>	23.29
<i>Large error samples (≥ 70)</i>	0.6%
<i>Steering saturation ($+25^\circ$)</i>	0.0%
<i>Steering saturation (-25°)</i>	0.0%
<i>Zero-error samples</i>	20.1%

Through iterative testing, the PID gains were refined from their MATLAB-estimated initial values to $K_p = 0.18$ and $K_d = 0.03$. The deadband and error smoothing buffer were also tuned on the physical track, as simulated responses did not capture surface friction, sensor noise, or the mechanical backlash of the servo. Testing was conducted on the competition track to replicate the specific lighting and surface conditions as closely as possible.

6.3.3 Turn Execution Validation

Turn performance was validated by placing the vehicle at multiple starting positions within the same turn type and observing whether the exit lane was successfully re-acquired. For trajectory-guided turns, the camera tilt angle, feedforward gain, and minimum line-check timer were iteratively adjusted until a consistent exit was achieved across at least X successive attempts per turn direction. The minimum line-check timer was particularly sensitive, as too short a value caused the grayscale sensor to lock onto an adjacent lane line mid-turn, while too long a value caused the car to overshoot the exit lane. For outside-wheel turns, the PWM values and turn duration were similarly adjusted per direction. A blind turn fallback was also verified by blocking the camera feed and confirming that the car still completed the turn, albeit with reduced accuracy.

6.3.4 I/O Validation

Turn signal and brake light behavior were first verified in isolation using a dedicated test script that activated each LED independently and confirmed the correct GPIO pin response. Integration testing was then performed by manually commanding the mission state machine to each relevant state (LEFT_1, LEFT_2, RIGHT, APPROACH_STOP) and observing that the correct blinker activated and that the brake light engaged when the vehicle slowed before the stop line. Hazard mode (simultaneous blinking of both blinkers and the brake light) was verified during e-stop and duck-stop events.

Horn behavior was validated first through a standalone audio test script, then confirmed within the integrated system by triggering duck-stop and obstacle-stop events and verifying that the horn sounded on entry and repeated at the configured 3-second interval while the stop condition persisted.

6.3.5 Obstacle Detection Validation

Obstacle avoidance was validated by running the full control loop and placing a physical obstacle at varying distances in front of the vehicle. The ultrasonic median filter was confirmed to suppress spurious readings by rapidly removing and replacing the obstacle and verifying that the vehicle did not falsely trigger a stop on a single noisy reading. The vehicle consistently stopped when the filtered distance fell below the 10 cm threshold, activated hazard lights and horned, and resumed motion once the obstacle was removed and the buffer cleared.

6.4 Competition Results

The system successfully passed the pre-competition safety verification with a perfect score of 100%, confirming that all required criteria across physical design, safety systems, signaling, motion control, and electrical integrity were met in a controlled environment. This included successful validation of key functionalities such as stable lane following, proper obstacle stopping without contact, reliable emergency stop functionality, and fully operational signaling systems. As a result, the team began the competition with the maximum starting reputation of 30% and an initial balance of \$1000, aligning with expectations for a safety-focused and compliant system.

Despite this strong verification performance, competition results diverged significantly due to runtime limitations, power supply errors, and failures under dynamic conditions. Although the competition lasted 15 minutes, the vehicle was only operational for approximately 3 minutes. A major contributing factor and unexpected issue was an unstable SSH connection, which disconnected unexpectedly and required 5-6 minutes to reconnect. This significantly reduced the system's effective runtime, and explains why, despite claiming three fares, only one was completed and one was dropped. Additionally, while this issue was occurring, the team noticed power supply errors although the battery was fully charged and was supported by a fully charged portable power bank. Therefore, task completion metrics were constrained significantly by infrastructure and hardware reliability rather than pure system performance, since the completion of one successful fare indicates holistic system success.

From a quantitative standpoint, the team ended with \$989.02 and a reputation of -31%, driven by two standard and three severe safety violations. Two of the three severe violations occurred due to the vehicle turning out of the lane bounds and hitting a stop sign, which is believed to be a power supply issue which affected motor output since the vehicle was able to complete this turn consistently during testing. The third severe violation was an accidental rearending of another vehicle that was stopped at the beginning of a lane as the team's vehicle was turning right into that lane at a stop intersection. This revealed an unexpected edge case where since the ultrasonic sensor was mounted at the front of the vehicle, when it

is completing curved turns, it fails to detect vehicles immediately at the front of the lane the vehicle is turning into.

These results fall short of the original objective of maintaining a positive reputation and achieving system bonuses such as “Safety First” and “Spin Me Round.” These behaviours were not expected and contradict the design assumption that redundant sensing, such as from camera vision, would not be needed. Instead, the results indicate that while all required subsystems were present and functional, their performance was not robust under dynamic, real-world conditions. However, when accounting for reduced runtime and unexpected power supply issues, the system’s ability to claim and complete one successful fair indicates that core features and design requirements were functional when active.

7. Limitations, Risks, and Recommendations

7.1 Challenges, Limitations, and Assumptions

A major limitation of the team’s design is that it is only functional within the Quackston map. Many aspects of the map had to be hard coded into the system so that Control could make the correct decisions at the correct times. The hard coding was based upon the assumption of correctly taped roads, with intersecting tape on every intersecting path on the map. This dependence arose because of other limitations encountered during development. Originally, the team intended to use object detection for sign recognition, which would have allowed stop signs to inform upcoming stopping behaviour more directly. This would have been especially useful because Control navigates primarily using the grayscale sensor, counting the number of “triple detections” it encounters and using that count alongside the queued directions sent by Localization and Planning. However, Control has no inherent ability to distinguish between a true stop line and other taped intersection features that produce the same grayscale signature. As a result, information such as which graph edges contain stop signs and how many triple detections must be passed before a turn or stop decision is made had to be encoded directly into the map representation. In theory, reliable stop-sign detection could have reduced this hard-coding by informing Control when a valid stop line was approaching. In practice, however, sign detection was not reliable enough. A single stop-sign misdetection could cause the vehicle to stop at the wrong triple detection, potentially in the middle of traffic, and would then throw off the instruction queue passed from Localization and Planning for the remainder of the route or at least until it updates.

Another limitation is the turning and recovery. The turning uses some hard-coded parameters that were tuned by hand and stored in a config file for different scenarios. These parameters would include things like PWM, absolute steering angle increases, and inner to outer wheel PWM ratios for turning. This can make for unreliable turning in certain niche scenarios causing the car to go off track. Although the CTE, produced by perception helps control in navigating the turns, if the produced output was obstructed by lighting or an uneven surface, this would also cause an incorrect turn. Uneven surfaces were a problem that arose, as the map is made from square cut sections that were pieced together but were not assembled flesh with the ground and each other. A flesh and flat assembly of mats was an assumption that the team had and heavily relied on for the CTE creation but did not hold. There was also not enough development time put into a reliable recovery algorithm and the team settled on a more rudimentary algorithm which compounds with the turning issues.

Unstable SSH connections also proved to be quite a challenge when it came to testing, as it was a large time sink, and created a failure mode that the team did not initially expect. The team’s ability to mitigate these connectivity problems is limited but it is worth mentioning.

7.2 Risks and Lessons Learned

The largest risk that was seen in the final system was dependence on single sensors. Lane following when going straight relied entirely on the grayscale sensor, this created a single point of failure for the entire vehicle. In a future iteration, CTE could be integrated at all steps of the control process and not just turns, this will increase reliability. The detection of the ducks relied entirely on the object detection model, and although this worked fine it still presented a risk as its height was too low to be detected by the ultrasonics. This single point of failure again could be avoided by installing extra ultrasonic sensors specifically for certain high importance objects like pedestrians. Another risk involves the reliance on high-speed communication between subsystems. All the subsystems relied on timely ZMQ messages and a consistent shared state, this creates risk of dropped messages or timing mismatches especially under load. To mitigate this risk, the team could have created application side acknowledgements before action is taken.

7.3 Societal and Environmental impacts

For this design, the main social trade-off is that safety and predictability are achieved by heavily constraining the engineering problem that is autonomous taxi cars. Although this design may be appropriate for Quackston, it would not be able to scale to wider deployment without major improvements or by applying constraints in the scaled environment. If scaled, it could replace jobs in the transportation business which could generate a negative perspective from society. The design also does not provide many accessibility considerations as the car was designed within the scope of identical ducks, leading to further societal infractions.

The team's philosophy of simple reliability coupled with the inherent reuse model that the course enforces makes for a minimized environmental impact. The reusability model and having minimal additional hardware due to the team's philosophy are positives, however there are environmental issues that come with scaling a project like this. If scaled with a large-scale deployment of the autonomous vehicle, this would cause a significant increase in electronics use, battery demand, and compute overhead creating an energy burden that may manifest in negative environmental impacts.

7.4 Recommendations for Future Iterations

Compiling ideas from earlier sections, future iterations should focus on reducing hard-coded dependence on the Quackston map. Important improvements in that direction would include stronger stop-sign detection to remove some of the burden currently placed on map creation. A more adaptive turn execution system, with fewer configured parameters and a more robust recovery mechanism, would also greatly improve vehicle performance. Reducing single points of failure should also be a priority, such as using the CTE at all times to assist the grayscale sensor in line following.

All of these changes are strong recommendations for a future iteration of the project. However, they would not be as useful if communication between modules remained unreliable. Increasing reliability in the communication network is necessary for a functional autonomous vehicle, so application-side acknowledgements should be added for key decisions, such as confirming whether a transmitted CTE is adequate for executing a turn.

8. Conclusion

This project successfully developed an autonomous taxi prototype capable of navigating the dynamic urban environment of Quackston. The system addressed the core challenge of integrating three interdependent systems, perception, planning and localization, and control, under strict hardware

constraints. The system achieved a perfect 100% score during pre-competition safety verification, demonstrating full compliance with all physical , electrical, and operational requirements. Although competition performance was hindered by unstable SSH connection, power supply inconsistencies, and edge-case sensor failures, which limited operation to 3 of the 15 available minutes, the successful completion of one fare confirmed that the core system architecture was fundamentally sound and functional when conditions permitted.

The most critical recommendation for future iterations of the project is securing access to the full competition mat well in advance of the competition, rather than the weekend before. Additionally, finalizing map and environmental conditions ahead of time is also recommended since late-stage changes to the mat, such as adding tape branches to all outside intersections, rendered months of perception algorithms and effort obsolete. Together, these two recommendations would prevent redesigning algorithms and recalibrating parameters under significant time constraints, and thus, enable more time for refinement and robustness testing. Overall, this project represents a meaningful achievement in resource-constrained autonomous systems integration, and while the competition outcome did not fully reflect the team's technological progress, the lessons learned in subsystem coordination, real-time control, and hardware reliability provide a strong foundation for continued engineering development.

References

- [1] Synopsys, "The 6 Levels of Vehicle Autonomy Explained," Synopsys Automotive, [Online]. Available: <https://www.synopsys.com/blogs/chip-design/autonomous-driving-levels.html>. [Accessed 4 April 2026].
- [2] D. J. Yeong, G. Velasco-Hernandez, J. Barry and J. Walsh, "Sensor and Sensor Fusion Technology in Autonomous Vehicles: A Review," 18 March 2021. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8003231/>. [Accessed 4 April 2026].
- [3] Precedence Research, "Autonomous Last Mile Delivery Market Size, Share and Trends 2026 to 2035," 18 February 2026. [Online]. Available: <https://www.precedenceresearch.com/autonomous-last-mile-delivery-market>. [Accessed 4 April 2026].
- [4] NHTSA, "Critical Reasons for Crashes Investigated in the National Motor Vehicle Crash Causation Survey," CrashStats, [Online]. Available: <https://crashstats.nhtsa.dot.gov/Api/Public/Publication/812115>. [Accessed 4 April 2026].
- [5] M. Green, "Accident Statistics on Driver Error: The 94% Myth," 2024. [Online]. Available: <https://www.visualexpert.com/Resources/drivererror.html>. [Accessed 6 April 2026].
- [6] C. Wink, "What a decade of ridesharing actually did to American cities," Technical.ly, 22 March 2026. [Online]. Available: <https://technical.ly/workforce/how-uber-resaped-cities-labor-traffic-builders/>. [Accessed 4 April 2026].
- [7] Y. Freemark, C. Stacy, O. Fiol and J. Morales-Burnett, "Regulations to Respond to the Potential Benefits and Perils of Self-Driving Cars," Urban Institute, September 2022. [Online]. Available: <https://www.urban.org/sites/default/files/2022-09/Regulating%20Autonomous%20Vehicles%20Report.pdf>. [Accessed 6 April 2026].
- [8] E. Richter, "The Precautionary Principle, epidemiology and the ethics of delay," PubMed, [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/15212202/>. [Accessed 6 April 2026].
- [9] "Consequentialism," Stanford Encyclopedia of Philosophy, 4 October 2023. [Online]. Available: <https://plato.stanford.edu/entries/consequentialism/>. [Accessed 6 April 2026].
- [10] Electrical and Computer Engineering Department, "Gitbook: ELEC392 Project," [Online]. Available: <https://elec392.gitbook.io/elec-392/8Ty9Nx3l84dHok0cVgCY>. [Accessed 4 April 2026].

Appendix A: Team Collaboration

Responsibility Distribution and Management

The team organized development around three primary subsystems (Control, Perception, and Planning and Localization), each owned by one or two members with cross-team collaboration as integration demands required. These responsibilities were assigned to people based on their personal interests and strengths. This division was established collaboratively during the early planning phase, with each member self-selecting their area while the team validated that the workload distribution was equitable before committing. Below is the subsystem breakdown:

Table 3: Breakdown of responsibilities per subsystem.

Subsystem	Primary Owner(s)	Key Interfaces
Perception	Nolan, Ishaan	Control
Planning and Localization	Declan	Control
Control	Rafael	Perception, Planning and Localization
Systems Integration	All members	All subsystems

Primary ownership meant accountability for design decisions, implementation, and testing within that subsystem. However, ownership was not treated as exclusive as other members regularly reviewed each other's code and participated in integration sessions that crossed subsystem boundaries.

Workload Equity

Task delegation was managed through a shared GitHub Kanban board, where work items were created, assigned, and tracked to completion. This gave all members visibility into progress across subsystems and made it straightforward to identify when one member was overloaded, and tasks needed to be redistributed. For example, during the final integration sprint, the complexity of adding new map definitions to the planning system proved larger than anticipated. Nolan, Ishaan, and Declan then worked together rather than leaving it solely to the subsystem owner.

Leadership rotated naturally depending on the phase of the project. Early on, subsystem leads drove their own design decisions independently. As integration became the central challenge, coordination responsibilities shifted toward whoever was managing the blocking dependency at a given time. Weekly meetings kept all members aligned and provided a recurring chance to raise concerns, reprioritize tasks, and make joint decisions when subsystem boundaries were ambiguous.

Decision-Making and Conflict Resolution

When there were decisions that arose, the sub-team that was responsible for the decision would submit a GitHub issue which describes the problem. This is important as it makes all aspects of the proposal comprehensible for all members including those outside the sub team. A discussion would be held which evaluates the solution on metrics like safety impact, scoring impact, implementation complexity, budget constraints and testability. Following this discussion a vote will take place where each person can present their ideas and their justification for their vote. If there is a tie, the team leader will have the final say. An example of decision-making using this method was when the Perception team had to decide how filtering for ducks was going to work and to theorize different ways to stop out of lane ducks from being detected.

Both members of the Perception team presented their ideas, and the team discussed which option would be best for the project, keeping the metrics in mind.

Conflicts were resolved with an evidence-first based approach where those involved would submit written arguments within the issue. Each proposed solution will have measurable results, and small experiments may be used to argue for or against solutions.

What Worked Well

Generally, workload equity was quite balanced, and even if there was a discrepancy, members of the group who had less work at that time would shift focus towards other sub-teams. An example of this is when perception had finished the majority of image processing for CTE, and also object detection, both members of that sub-team shifted focus to help with control and annotating map features for localization and planning.

Decision making and conflict resolution was also a big success in the project. Rather than taking the very structured approach described above, most of the time quick in-person discussions were adequate for deciding a design direction. This got rid of the need for opening issues on GitHub as issues would be discussed in our meetings that were held twice a week.

Appendix B: Individual Reflections

Rafael

My main role in the project was developing and maintaining the control system, which was the lowest level system of the vehicle. This included the PID-based lane-following controller, the mission manager and state machine, turn execution logic, obstacle avoidance, and the TCP communication layer connecting control to both the perception and pathing subsystems.

The size of the role forced me to develop a more systems-level approach that I did not initially have coming into the project. Early on, I approached the control system as a self-contained issue. I was tuning the PID gains, handling intersections, that's it. What I then learned was how tightly coupled control is to every other subsystem. The quality of the perception team's CTE output directly determined whether my trajectory-following turn logic would succeed. The format and timing of the pathing team's direction messages shaped how the mission manager had to be structured. I found myself spending a significant amount of time designing not just the control logic itself, but the interfaces between systems. This included the TCP message schemas, timeouts for information, and a fallback system for when data received was stale or missing.

I also developed much stronger skills in real-time software design, PID tuning under the physical hardware constraints, and programming in a way that anticipates errors that may come up. I noticed that writing code that runs in a loop on the Pi, interacts with physical sensors and servos must handle sensor noise, timing jitter, and unexpected states. This taught me habits that I often did not get from classroom projects. I became more deliberate about things like e-stop interrupts and specific timings for loops so that if there is an issue, the code either recovers cleanly or fails safely.

A key gap I identified in myself is the tendency to over-engineer solutions before validating simpler ones. Several times during this project I built relatively complex mechanisms like the trajectory-based turn system before adequately confirming that a simpler approach was insufficient. In some cases, though, complexity paid off; in others it just introduced fragility and more debugging. This consumed time that would have been better spent elsewhere. Going forward, I would apply a stricter principle of providing a minimal version that works first before layering in additional sophistication on top.

A related lesson would be the timing of the integration of subsystems. Because I owned so much of the control system, it was tempting to keep iterating it in isolation, trusting that integration with perception and pathing could happen later. In practice, the bugs that ended up mattering the most, like the necessity for extra turn states, broke under real driving and only appeared once the full system was running together. Earlier and more frequent testing, even with simple implementations of other subsystems, would have surfaced these issues sooner and reduced last-minute pressure. This is something I would prioritize differently if I repeated the project.

Ishaan

Throughout this project, I was primarily responsible for designing and implementing the perception subsystem, with a focus on lane detection, cross-track error (CTE) estimation, and object detection integration. My contributions included developing the full lane detection and CTE pipeline using a bird's eye view (BEV) transformation, implementing robust stop-line detection with multiple validation checks, and assisting Nolan with the design of the overall duck detection pipeline with multiple filtering stages to reduce false positives. I was also responsible for the perception communication module using ZMQ, optimizing it with persistent sockets and non-blocking calls to minimize latency and ensure real-time performance. Beyond this, I assisted Nolan with object detection model data collection and annotation,

the wiring and software for the signaling system, and the construction of the CARE. Alongside these responsibilities, I assisted Rafael with parameter tuning, ensuring that CTE efficiently integrated with control, and helped Declan with adding unforeseen edge cases to the planning map. Beyond implementation, I played a key role in system-level design decisions, including refining scope and contributing to subsystem integration strategies.

Through this work, I developed strong technical skills in computer vision and image processing, including colour space transformations, morphological filtering, perspective warping, and polynomial curve fitting for trajectory estimation. I also gained practical experience in deploying machine learning models, particularly in dataset curation, annotation (Pascal VOC), and evaluating real-world performance tradeoffs such as accuracy versus latency. Additionally, I strengthened my understanding of distributed systems and communication protocols by implementing efficient communication using ZMQ and addressing latency bottlenecks. Debugging and understanding how different branches, loops, and modules interact and influence one another helped improve my analytical thinking and real-time problem-solving skills in high stress situations.

A key gap I identified in my approach was a tendency to over-engineer solutions when simpler alternatives were sufficient or only required partial modifications. This became evident in the duck detection pipeline when addressing false positives from ducks outside the vehicle's path. While an initial trapezoidal mask effectively constrained detections to the lane of travel, I later attempted to handle edge cases, such as round about entrances or T-intersections, by designing a complex, dynamic masking algorithm that adapted based on detecting lane boundaries and intersection features. However, I found that this approach added unnecessary complexity and increased latency. A much simpler and more robust solution, which was implemented, was to retain the original trapezoidal mask, but apply a portion of this complex algorithm to detect a forward white boundary and clip out irrelevant regions. Thus, this experience reinforced the importance of prioritizing simplicity and incremental improvements over overly complex designs, especially in time-constrained environments where valuable testing could be otherwise performed.

One of the most important lessons I learned was about the importance of making informed design tradeoffs based on system constraints rather than ideal theoretical performance. For example, narrowing the object detection scope to only ducks significantly improved reliability and reduced computational load, which demonstrates the value of prioritization in constrained environments. Likewise, transitioning from initializing communication sockets per frame to a persistent connection highlighted how small architectural decisions can have huge performance impacts on larger pipelines. I also learned the importance of cross-team integration and communication, since challenges such as synchronization issues between planning and control emphasized the need for clearly defined interfaces and shared assumptions early in the design process.

In future projects, I would place greater emphasis on early system-level validation, including testing perception models under realistic runtime conditions and aligning subsystem interfaces before full implementation. I would prioritize iterative integration earlier in the project timeline to identify dependencies and edge cases sooner. Lastly, I would allocate more buffer time in initial project management Gantt charts, to better allocate time for debugging and unprecedented delays and challenges. Overall, this experience not only strengthened my technical abilities, but also my collaborative skills and communication, particularly in working across multidisciplinary teams and coordinating between tightly dependent subsystems to achieve reliable system-level performance.

Nolan

My primary role in this project was on the perception team. I had a focus on the object detection and duck filtering side, while also contributing to CTE and overall perception brainstorming. I collected and labelled the duck dataset, implemented the colour-based filtering which eliminated a large number of false negatives. I also assisted in simplifying the perception code base as there were a number of complexities that were unnecessary and just ended up slowing the system down without any benefits. In addition to this sub system work I contributed to brainstorming overall for integration of the sub systems and thinking of the problem not only from my assigned sub system lens but how they would all fit together.

I developed stronger skills in computer vision, model training, debugging perception pipelines, and how to integrate these perception outputs with other systems like control. Initially I had a much narrower view of perception thinking that creating the model was the hardest part, but it is only the first step of building a usable perception system. Even with a strong duck detection model there is still a lot of processing that has to go into the detection to filter out false negatives and other undesirable scenarios. This led me to improve at practical filtering, and thinking about perception in the context of the competition and what is happening practically rather than just the isolated subsystem.

A large part of my learning also came from thinking about what would be the most useful and concise way to deliver information to another subsystem. I've improved greatly at communicating with the person on the other subsystem team, identifying what I can deliver and what they would like, then striking a balance. Communication when building an integrated product like this can be easily overlooked but it is important to understand at a high level the constraints of your peers so everyone can design around them.

A large gap that I have identified is that I tend to procrastinate on tasks that seem like they can be fixed in a short time period. The reason I would do this is because I think internally that it's a small fix and I can do it any time. I ran into this problem with object detection, when we were having difficulty with the duck detection early on, I could have easily just taken some more pictures and improved its quality. The reason it is important to do this early on is so that I can be exposed to the problems that are revealed afterwards. As I mentioned earlier, the detection wasn't the end of the road, there was still processing to do but I couldn't see these problems yet as the one in front of me was still the basic duck detection. It was only after duck detection was solid that we realized there were other circumstances we needed to engineer around, like the opposite laned ducks, and the off-road ducks who still appear in frame.

Declan

During this project I was primarily responsible for the Planning and Localization subsystem. Within the team GitHub, I am responsible for all the code and files within the fare-selection-pathing folder. Coming into the project, I had a reasonable degree of familiarity with graphing and graphing algorithms, so the team agreed it made sense for me to work on developing the code to direct the car.

I learned a great deal through the course of the project, through process of trial and error trying to develop a working set of functions to direct the car. A big theme for me through the course of development was trying to make forward looking decisions to set myself up for future success. I think a big issue I have had in past projects is that I just sit down and start working without giving a lot of thought to how the design decisions I make will expand or shrink my options going forward. This change in philosophy certainly paid dividends through the course of the project. For example, during brainstorming with Raphael (the control lead) early in the project, we were unsure exactly if I should send him instructions as x,y way points, left, right, straight directions at intersections, or some other option. I started working on the graph with this in mind, thinking through how I could work each option under the existing architecture if necessary. When

he eventually decided it would be most convenient for him if I gave him left, right, straight instructions at intersections, I did not have to go back and redesign my architecture to accommodate his decision.

The majority of my time spent on the project was spent improving on and fixing bugs in the pathing code. I will not go into detail on all of them as that would fill a lot more than one page, but I think one that sticks out in my mind as a good catch was when I was having consistent issues with classifying turns both at the start and the end of the car's route. When I looked at the graph to show where it should be travelling, I could see that it was selecting the correct nodes for the path, but the turns it was recommending were not matching what I saw on the graph. I realized that the car's heading was not being taken into consideration by Dijkstra's algorithm, which was causing it to plot impossible paths that included 180 degree turns. To fix this, I changed my code so that temporarily inserted nodes (representing the car's position or special points like the passenger / drop-off locations) would not include nodes in their adjacency list directly behind the car. To my surprise this did not fix the bug. On further reflection, I realized the car was simply travelling to the nearest intersection and performing a 180-degree turn, because I had removed the problematic node only from the temporary nodes adjacency list, but not the intersection.

These subtle problems highlight another major theme for me in this project, which is that bugs arise when you read the code and believe it is doing a certain thing, while in fact what you are thinking is not being precisely matched by what the code is doing. For this reason, I often found the best way to debug code was to walk through line by line and establish how exactly each piece of code contributes to the overall vision I have for that method / function and thereby find the points where those two things diverge.

Overall, this was a great project that gave me lots of opportunity to develop my planning, programming, and communication skills to build towards a successful final product. I am happy with my personal performance as I was able to build a system of algorithms to accurately move the car from start to destination consistently and output those directions to the control subsystem. I learned a lot during that process and am excited to complete another project like it in the future.

Appendix C: High Level System Overview

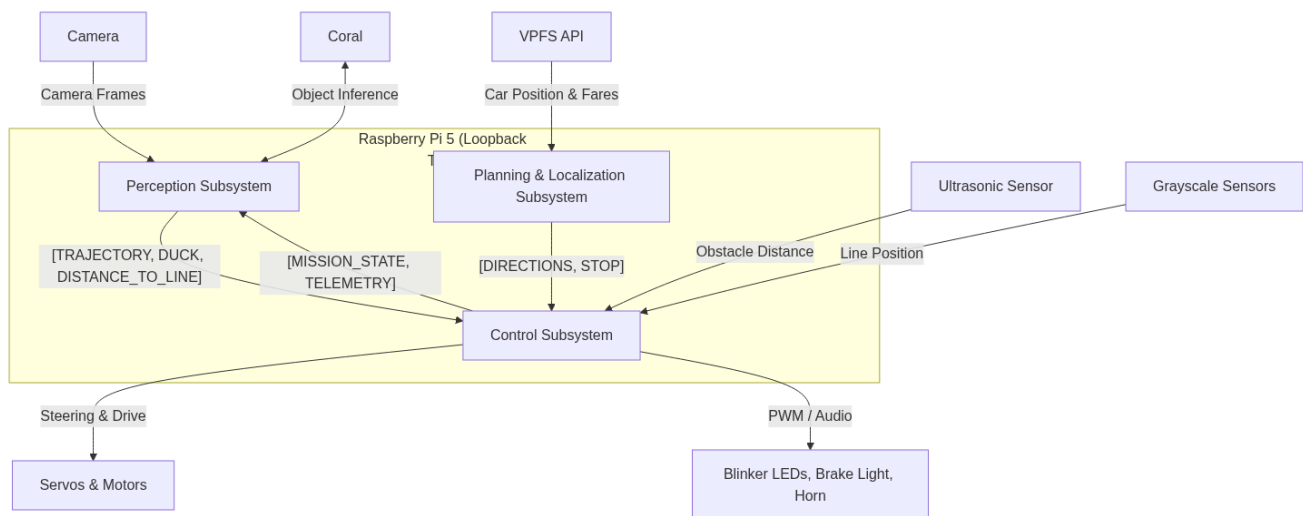


Figure 1: Flowchart depicting a high-level overview of the subsystems.

Appendix D: Planning and Localization

$$\frac{BASE\ FARE + (DISTANCE\ BASE\ FARE * FARE\ DISTANCE)}{DISTANCE\ BETWEEN\ CURRENT\ POSITION\ AND\ FARE\ COMPLETION}$$

Equation 1: Fare Score Equation

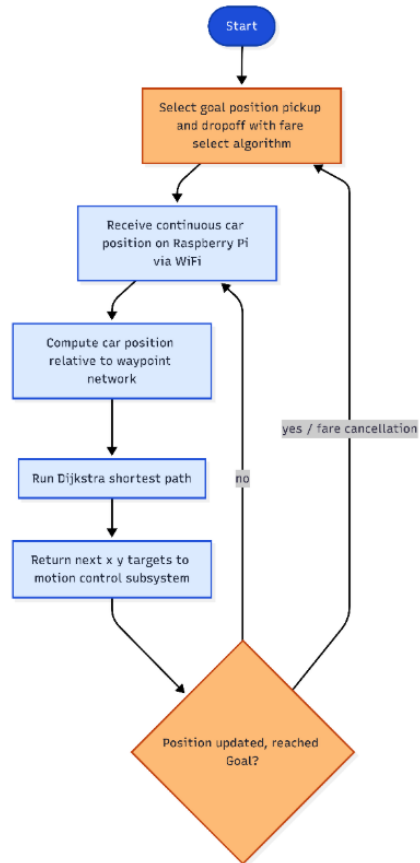


Figure 2: Path Selection Flow Chart

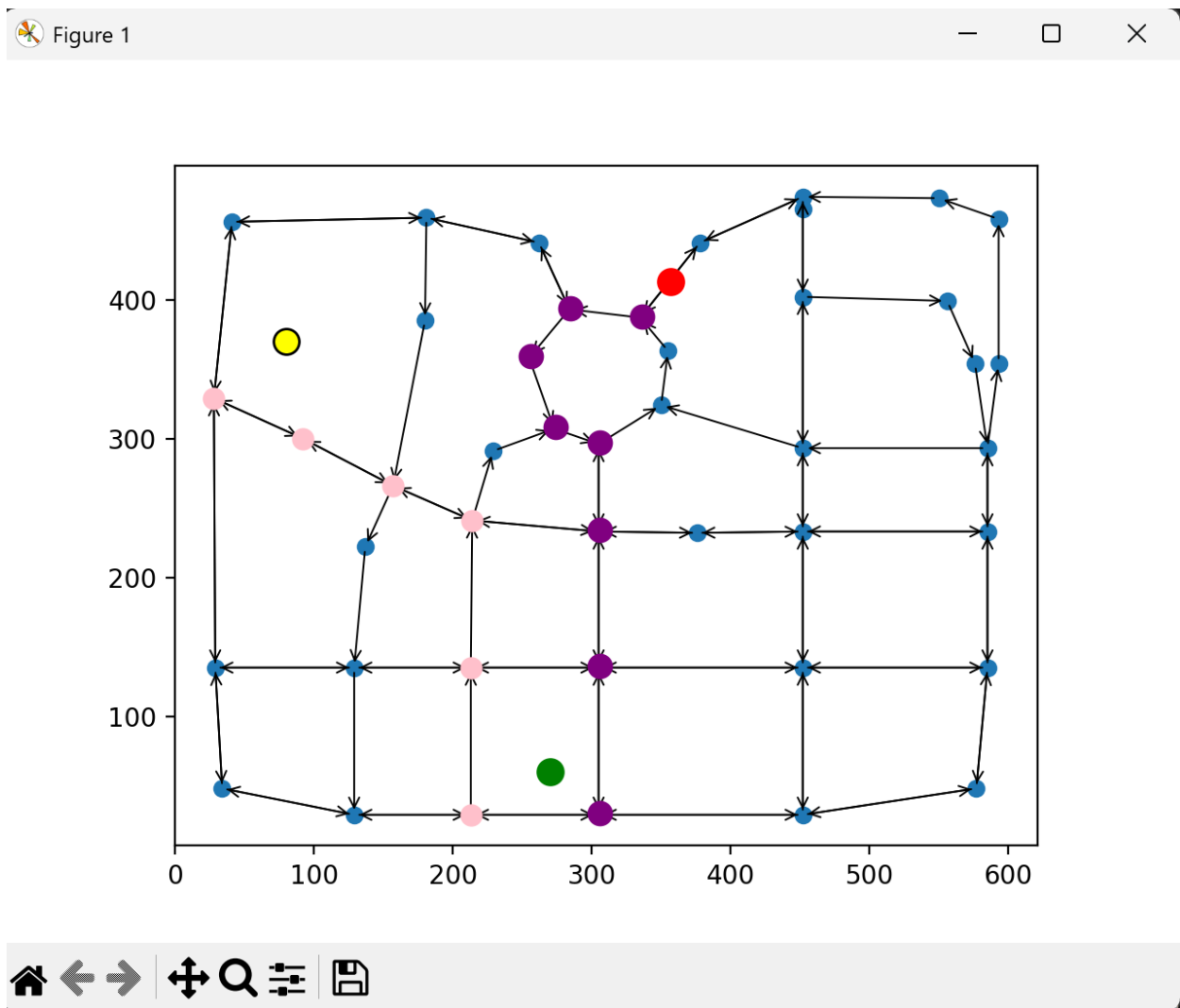


Figure 3: Example Fare Routing

Appendix E: Perception

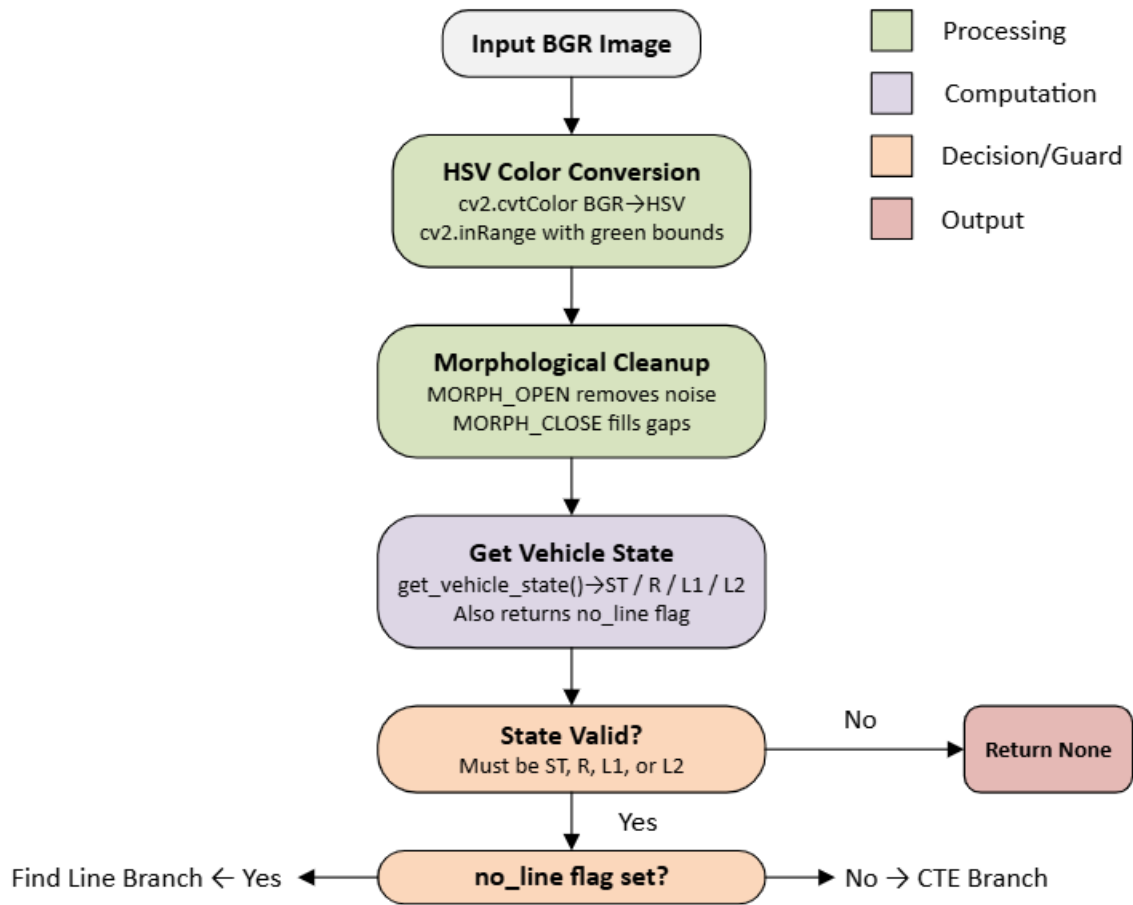


Figure 4: Perception Lane Detection & CTE Pipeline Flowchart.

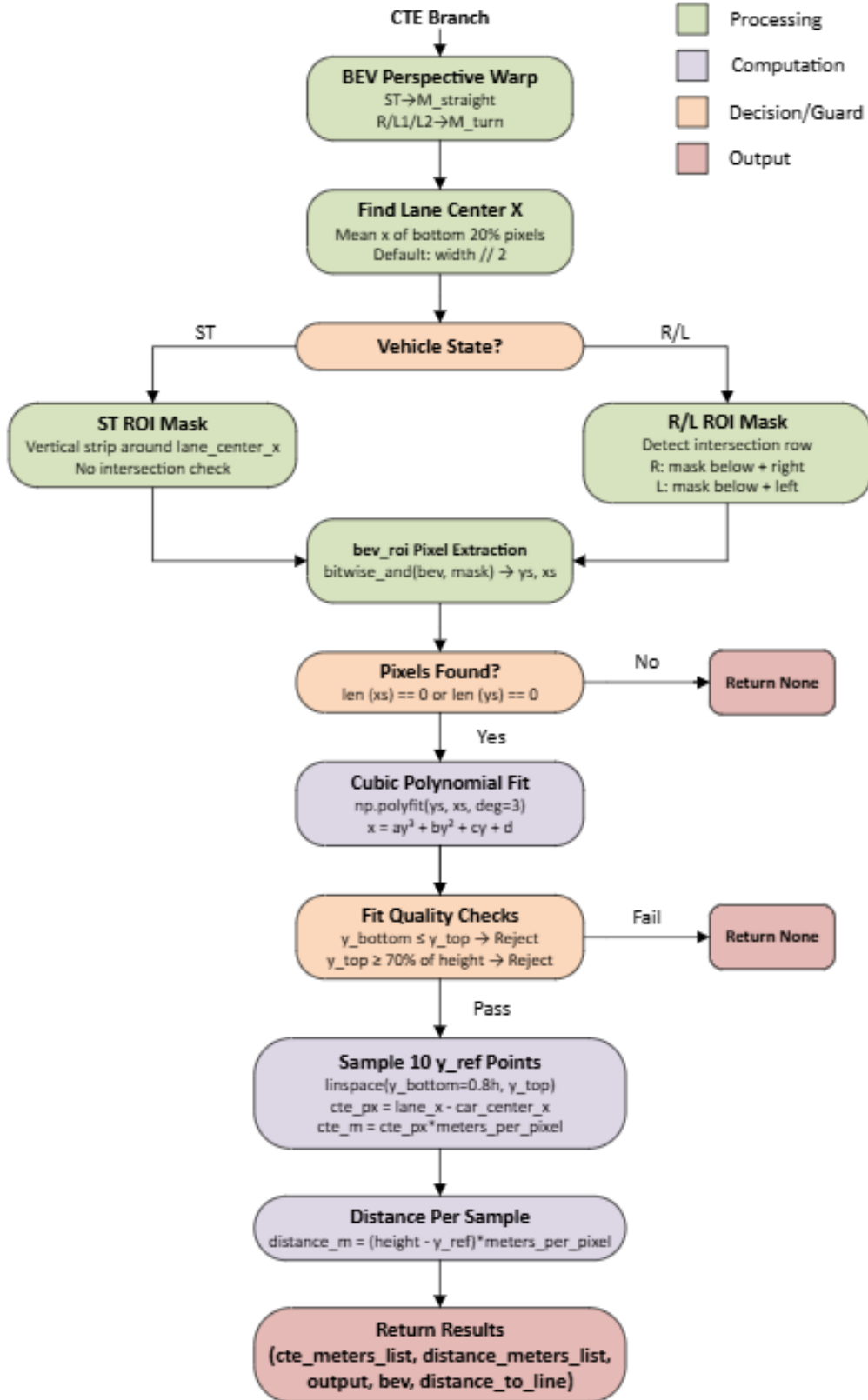


Figure 5: Perception Lane Detection & CTE Pipeline – CTE Branch Flowchart.

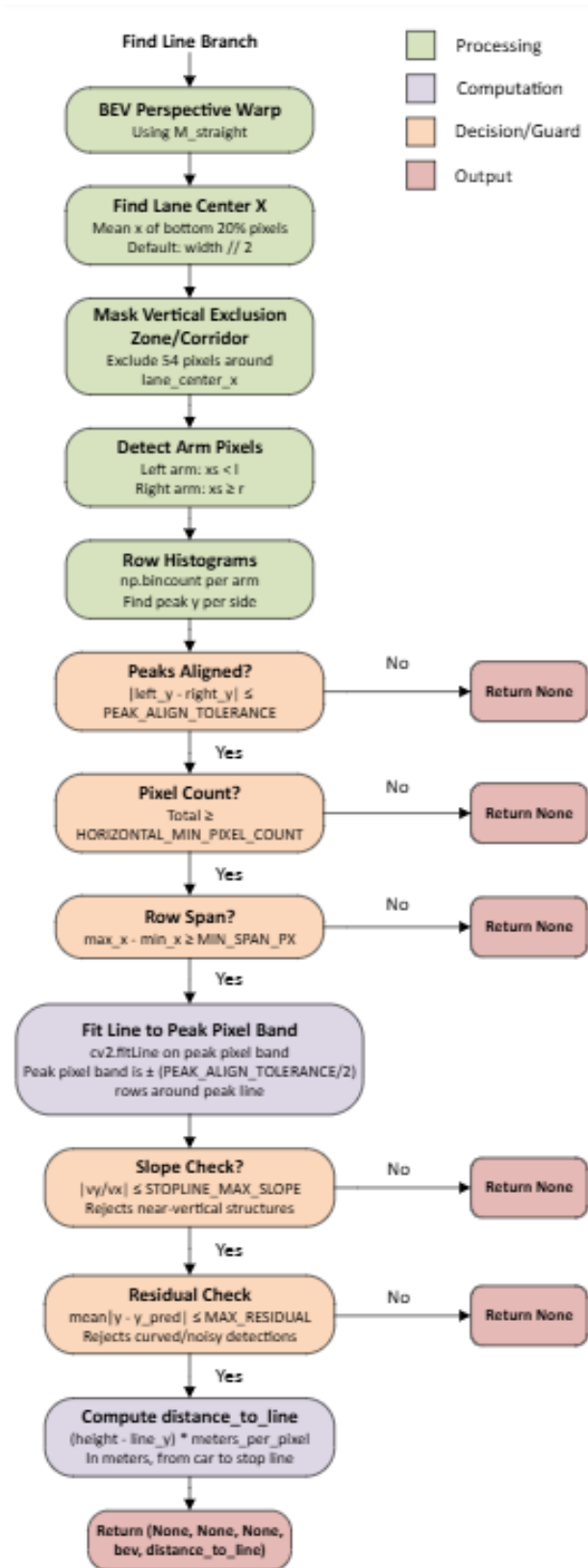


Figure 6: Perception Lane Detection & CTE Pipeline – Stop-line Detection Branch Flowchart.

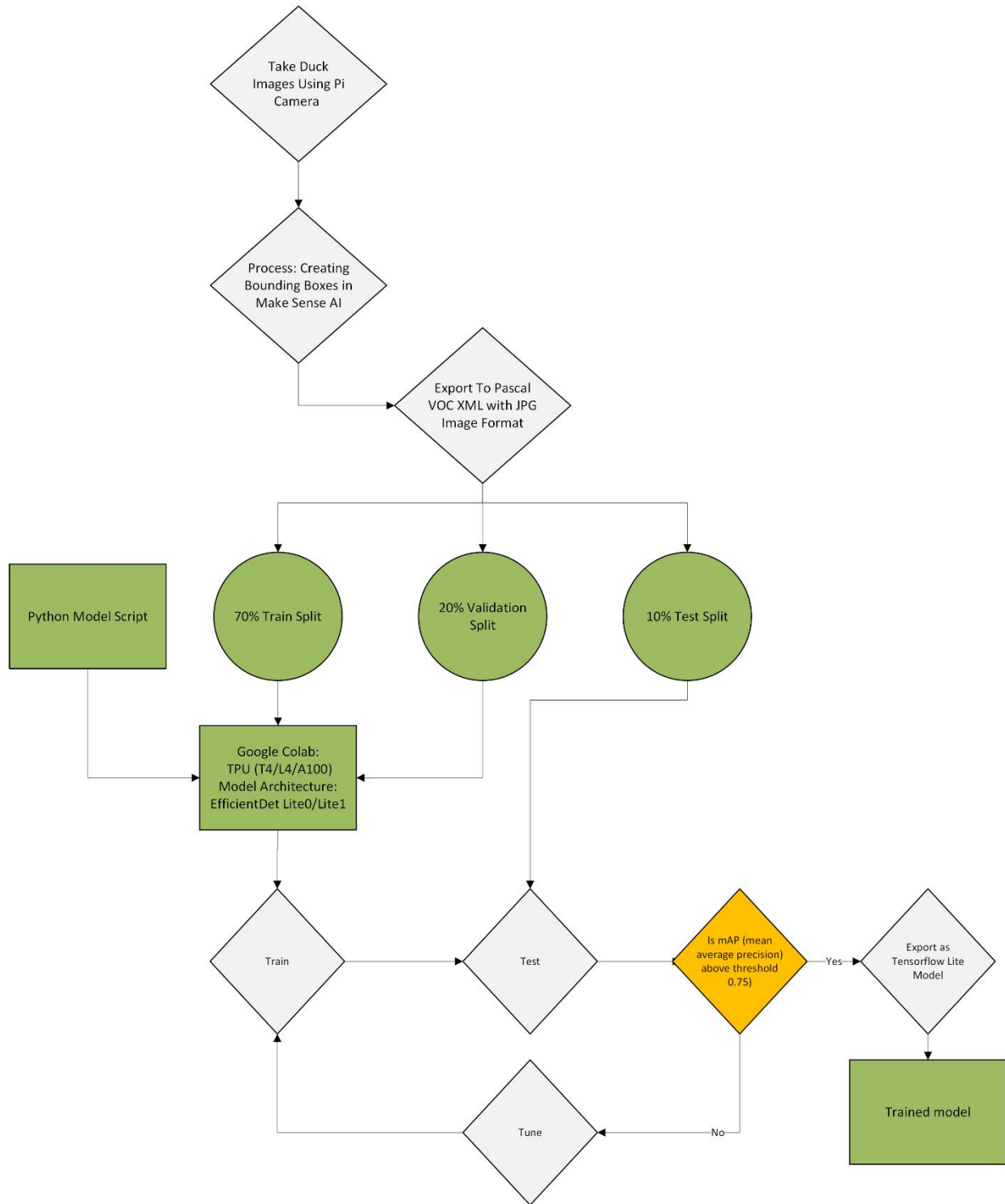


Figure 7: Perception Object Detection Model Training Pipeline

Table 4: Perception Object Detection Model Evaluation Results

Metric	Result
mAP (Overall AP)	0.797
AP @ IoU = 0.5	1.000
AP @ IoU = 0.75	0.971
AP (Medium Objects)	0.813
AP (Large Objects)	0.770
mAR (ARmax100 – Max 100 detections)	0.870
AR (Max 10 detections)	0.866
AR (Max 1 detection)	0.713
AR (Medium Objects)	0.882
AR (Large Objects)	0.843

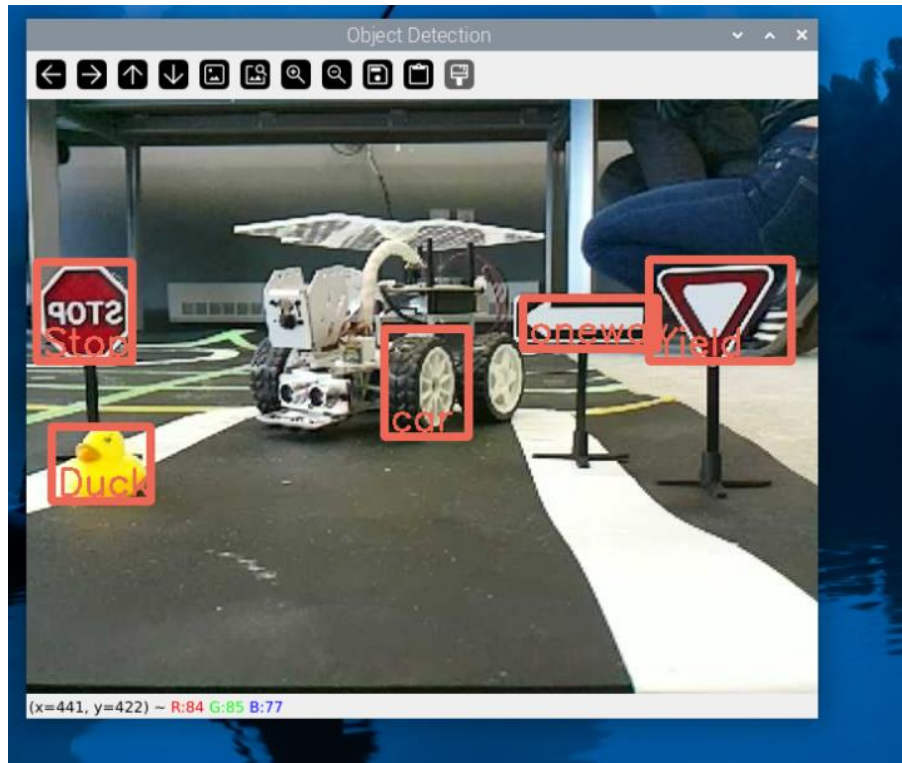


Figure 8: Object Detection Test Across All Vehicle Classes

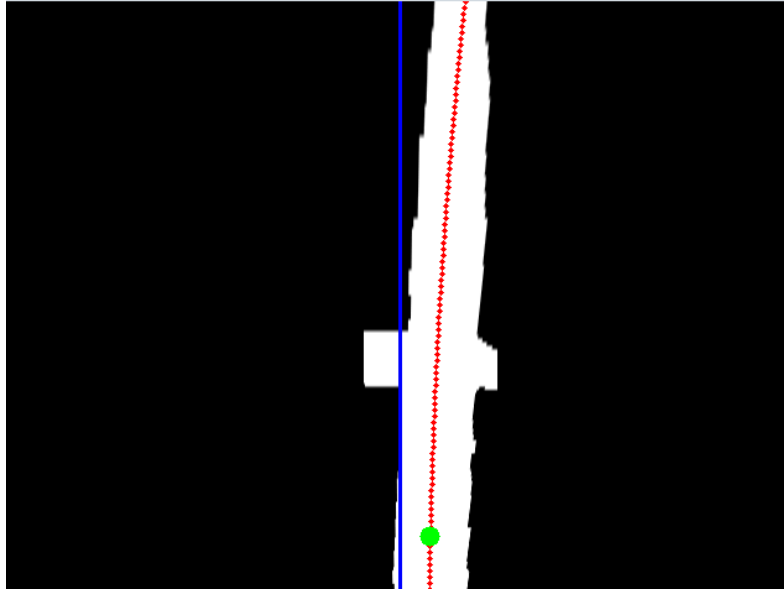


Figure 9: CTE Implemented for Straight Vehicle State. Red curve displays trajectory, green dot indicates look ahead point where CTE is calculated.

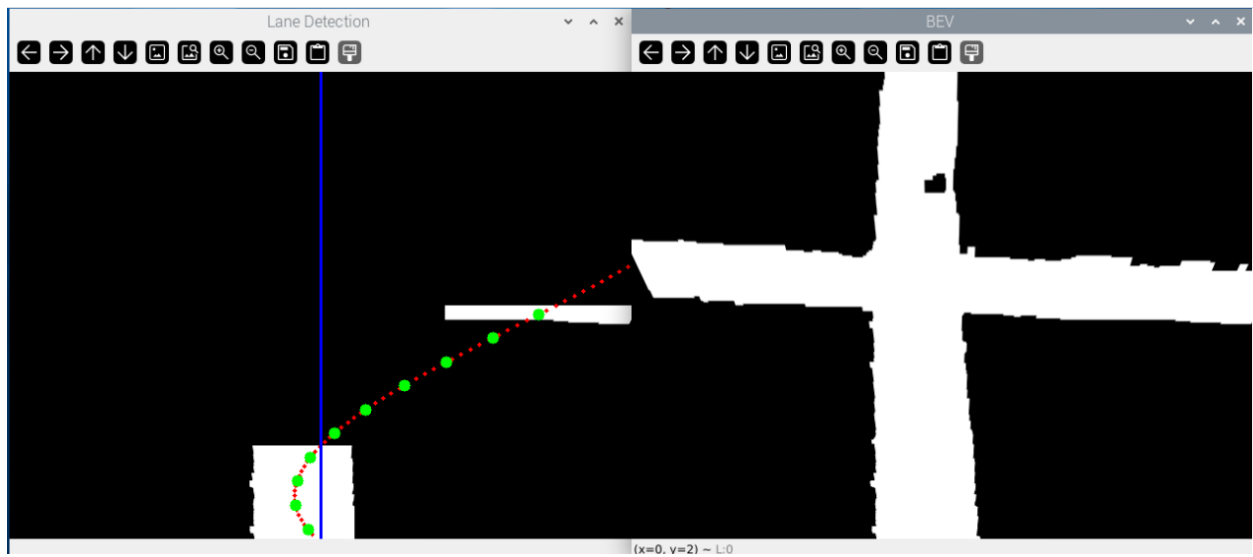


Figure 10: CTE Implemented for Turns. Red curve displays trajectory; green dots indicate 10 look head points where CTE is calculated.

Appendix F: Control High Level Overview

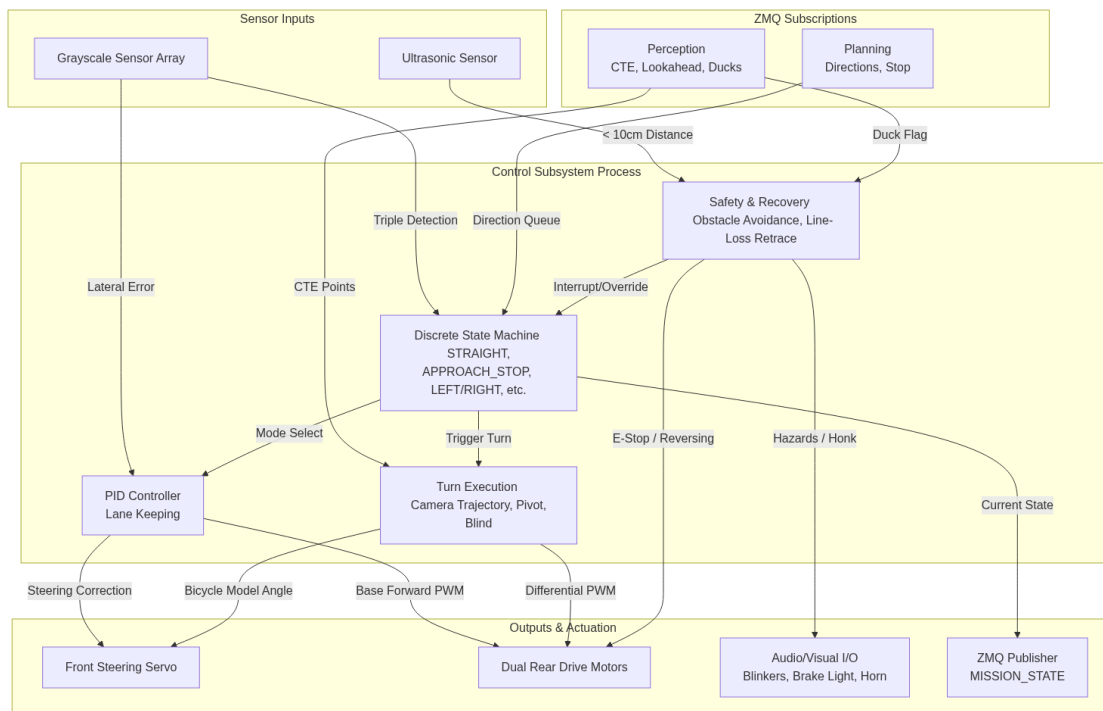


Figure 11: Flowchart depicting a high-level overview of the control subsystem.

Appendix G: List of Materials

Table 5: List of components used on the Picar.

Component	Role
Raspberry Pi 5	Central compute; runs all three subsystem processes
PiCar-X Platform	Mechanical base, servo steering, dual drive motors
Raspberry Pi Camera	Lane detection, object detection, trajectory generation
Google Coral USB TPU	Real-time object detection inference
Grayscale Sensor Array (×3)	Lane position feedback for PID control
Ultrasonic Range Sensor	Forward obstacle distance measurement
PWM Blinker LEDs (×2)	Turn signal and hazard indication
Brake Light LED	Braking and stop indication
Speaker / Audio Output	Horn for duck and obstacle events

Appendix H: MATLAB PID Tuner Image

Step Plot: Reference tracking

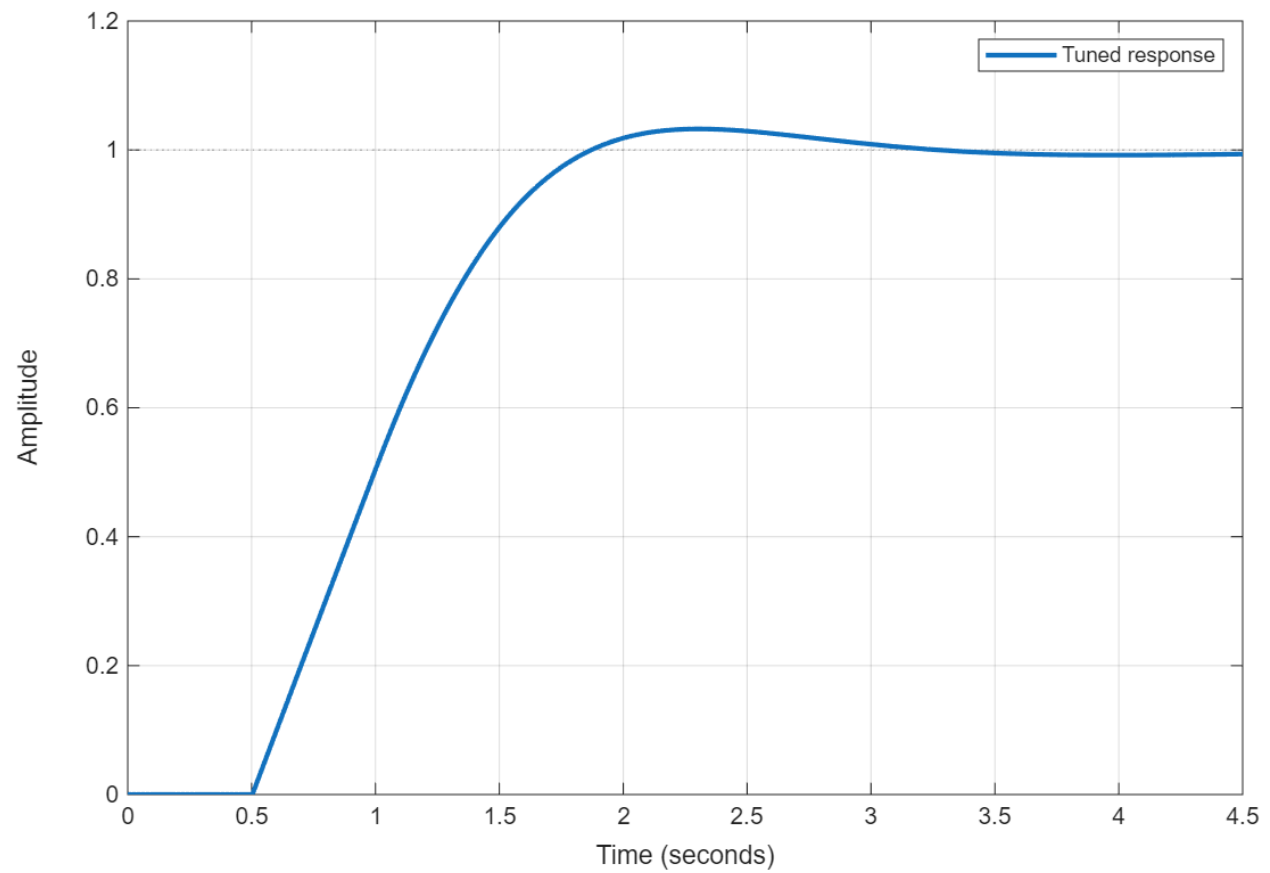


Figure 12: Screenshot of the MATLAB PID Tuner response curve.

Appendix I: Photos of the Picar

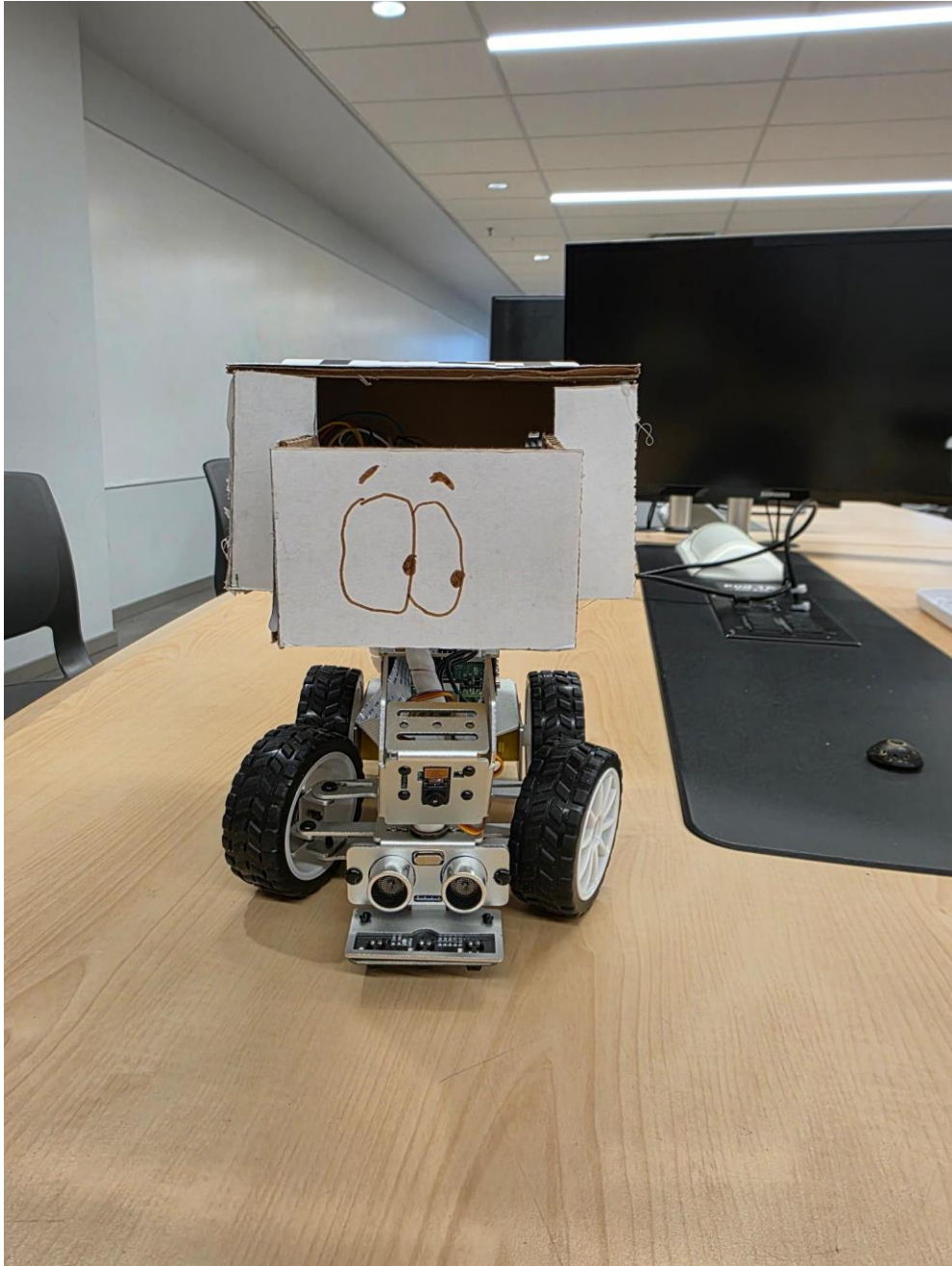


Figure 13: Front facing picture of Picar.

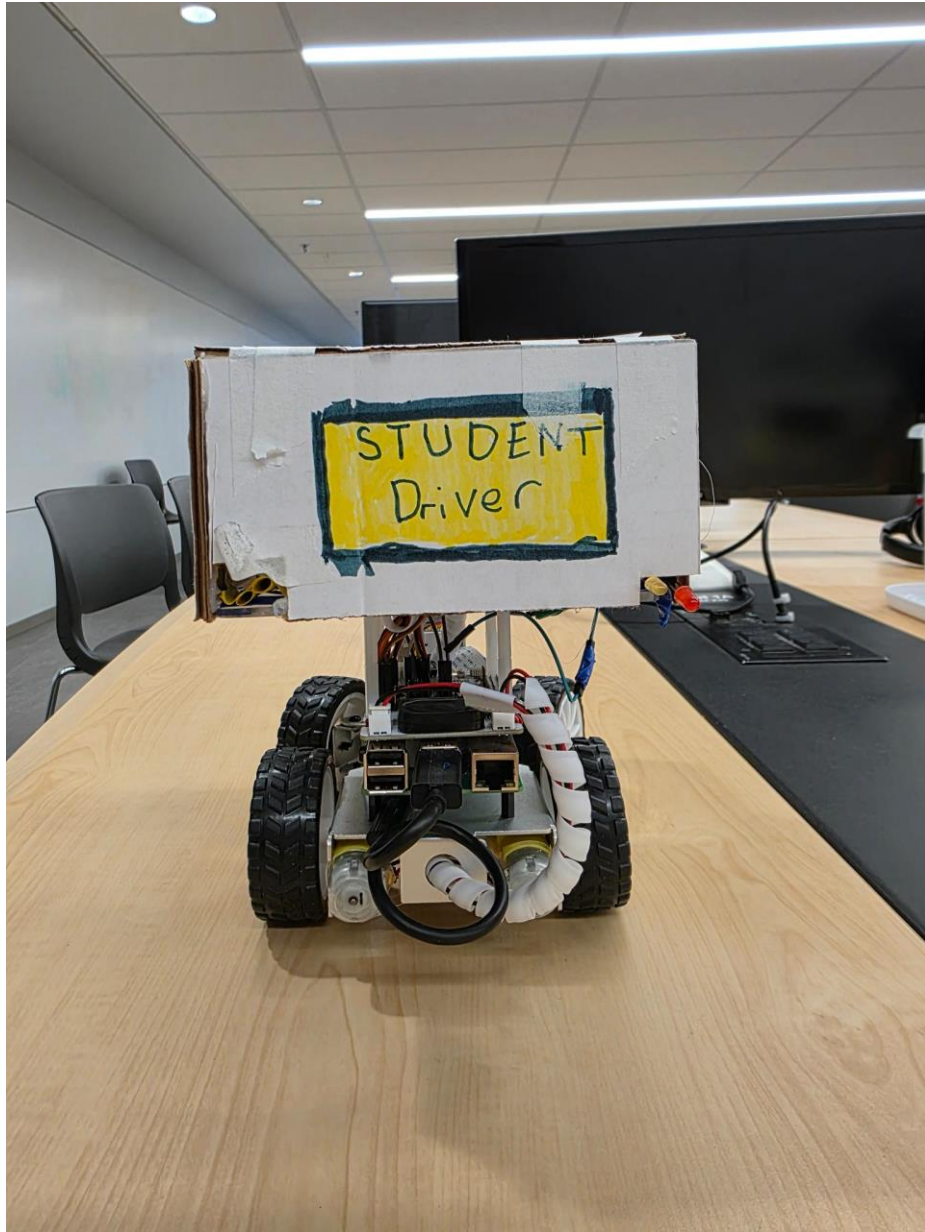


Figure 14: Back facing picture of Picar.

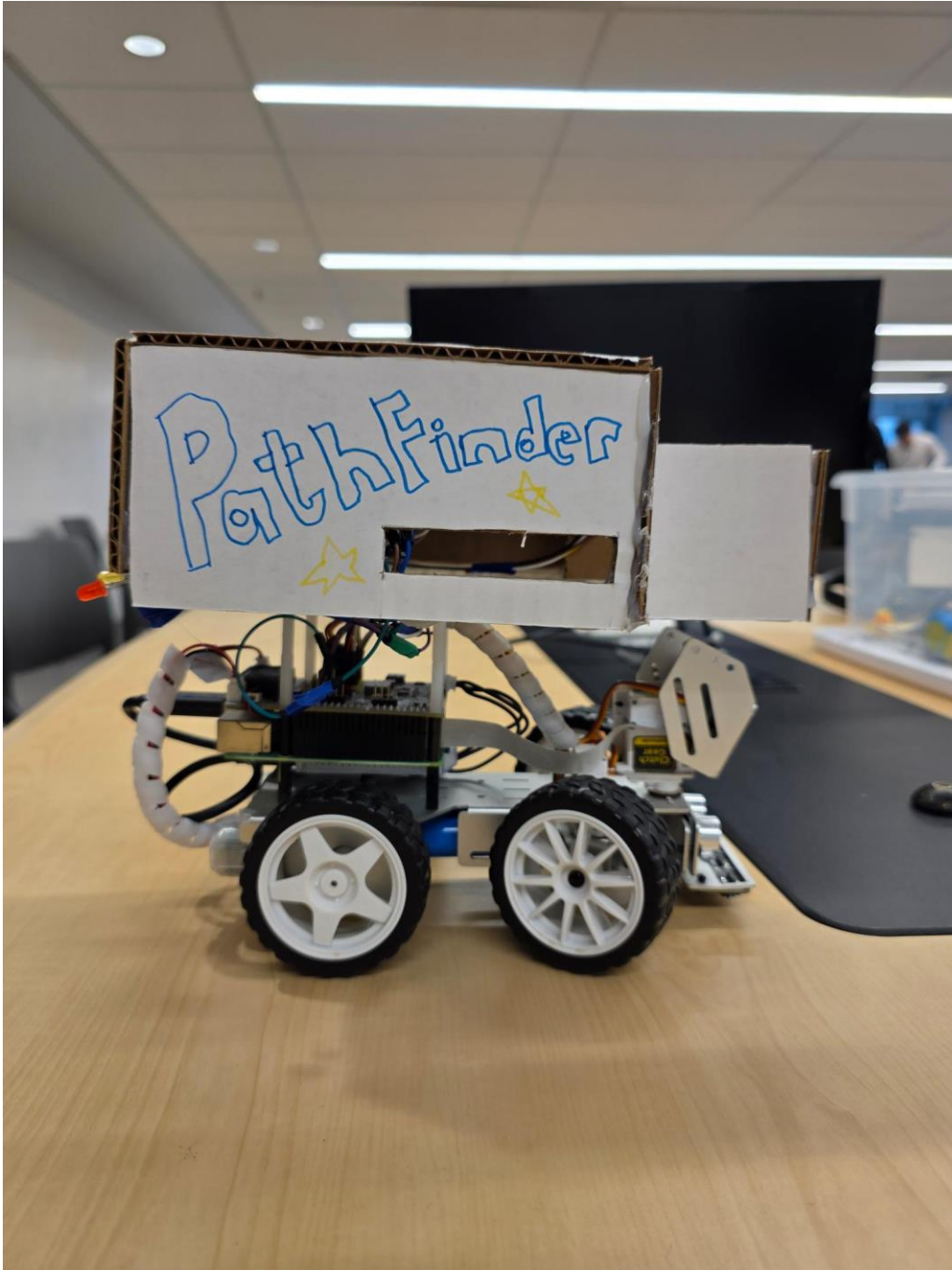


Figure 15: Side facing picture of Picar.



Figure 16: Top facing picture of Picar.